
DITRON: Distributed Multi-level Tiling Compiler for Parallel Tensor Programs

Size Zheng^{1,2} Xuegui Zheng¹ Hanshi Sun¹ Qi Hou¹ Wenlei Bao¹ Shiyu Li^{1,3} Haojie Duanmu^{1,4} Jin Fang¹
Chenli Xue^{1,3} Chenhui Huang¹ Yuanqiang Liu¹ Renze Chen^{1,5} Ningxin Zheng¹ Dongyang Wang¹
Li-Wen Chang¹ Liqiang Lu³ Yun Liang⁵ Jidong Zhai² Xin Liu¹

Abstract

The scaling of large language models is bottlenecked by the rigidity of distributed programming: high-performance libraries like CuBLAS and NCCL lack flexibility for evolving architectures, while existing tensor compilers fail to address the complex memory hierarchy of distributed clusters. We propose DITRON, a scalable tile-level compiler that introduces a hierarchical programming abstraction spanning Core, Device, and Task levels, enabling efficient mapping of tensor programs onto heterogeneous distributed hardware with diverse parallelism strategies. Evaluated across large-scale clusters, DITRON achieves performance parity with or exceeds expert-tuned CUDA libraries, delivering 6%–30% speedups on isolated kernels and 5%–30% on end-to-end inference, with strong portability across both NVIDIA and AMD platforms. Deployed at enterprise scale, DITRON improves training MFU by over 10%, which saves approximately 500,000 GPU hours per month and delivers over 20% end-to-end gains for inference across cloud and edge scenarios.

1. Introduction

The rapid evolution of Large Language Models (LLMs) places immense pressure on the underlying distributed systems. Only with high-speed, scalable distributed execution can models with diverse and emerging architectures be practically deployed (Dubey et al., 2024; Comanici et al., 2025; Rivière et al., 2024; DeepSeek-AI et al., 2024b; Qwen-Team, 2024; OpenAI, 2023; Anthropic, 2024). Historically, optimization efforts have been bifurcated: compilers like Triton (Tillet et al., 2019) and TileLang (TileLang-Team, 2025) focus on single-device kernel optimization,

while distributed communication relies on rigid libraries like NCCL (NVIDIA, 2024) or domain-specific communication compilers (Hwang et al., 2026). However, as cluster scales expand, communication overhead has emerged as the dominant bottleneck—accounting for 20%–80% of runtime in training and inference (Chang et al., 2024). This shifts the burden onto researchers, who must now possess the dual capability of designing novel algorithms and implementing them as highly optimized distributed kernels to prove scalability.

To address this challenge, the community has largely relied on two approaches: distributed Domain-Specific Language (DSL) compilers and distributed CUDA libraries. DSL compilers (Jangda et al., 2022; Google, 2025; Zheng et al., 2025) offer high-level abstractions for specifying parallel programs, while distributed libraries (NVIDIA, 2024; Chang et al., 2024; Zhang et al., 2025; Zhao et al., 2025) provide highly optimized, expert-tuned operators.

However, both approaches face significant limitations. First, flexibility is compromised. Distributed libraries are inherently non-programmable, blocking the exploration of new architectures. Meanwhile, DSL compilers often restrict users to operator-level granularity (Jangda et al., 2022; Wang et al., 2023), lacking the expressiveness for fine-grained optimizations. Second, scalability is often limited by rigid tiling assumptions. Frameworks like Pallas (Google, 2025) and TileLink (Zheng et al., 2025) enable tile-level programming but often lack a unified multi-level tiling abstraction necessary to pipeline execution across the complex hierarchy of a large-scale cluster. Third, the lack of portable primitives makes porting these frameworks to emerging hardware backends prohibitively expensive.

An ideal compiler should prioritize usability in its design without sacrificing performance. But this standard has not yet been achieved by any other existing work. In detail, we argue that distributed tensor programming should adhere to three core design principles. First, it should provide a flexible programming interface that aligns with the conventions of mainstream tensor compilers. As noted in a recent report (Liao et al., 2025), tile-level compilers such as Triton (Tillet et al., 2019) have surpassed CUDA

¹ByteDance Seed ²Tsinghua University ³Zhejiang University
⁴Shanghai Jiao Tong University ⁵Peking University. Correspondence to: Size Zheng <zheng.size@bytedance.com>.

to become the dominant kernel programming framework in the industry. Extending existing tile compilers to support distributed scenarios inherently holds advantages over developing a new programming language from scratch. Such extensions should ensure that legacy kernels can be converted to distributed versions with minimal code modifications. To this end, we adopt Triton’s tile programming model and implement distributed extensions atop it.

Second, it should embody a scalable programming paradigm that accommodates clusters of arbitrary scales and problem shapes. Hardware configurations vary significantly across different tasks. A distributed system may incorporate high-bandwidth interconnects such as NVLink, hardware-accelerated in-switch computing capabilities like NVSwitch, as well as low-bandwidth links including PCIe and Ethernet. Any a priori assumptions may render the compiler impractical for real-world system deployment. To enable seamless adaptation to such heterogeneous systems, we advocate for supporting multi-level tiling in the programming model, where fine-grained tiles are mapped to high-speed connections and coarse-grained tiles to low-speed ones.

Third, it should provide a unified set of primitives that is portable across diverse hardware backends. Different hardware backends feature distinct hardware topologies and underlying technologies. Consequently, low-level programming models entail drastically different optimization strategies and programming paradigms. To enable seamless support for multiple backends, we introduce a suite of hardware-agnostic primitives. Integrating a new hardware backend can then be achieved by instantiating these primitives and implementing corresponding translation rules for them.

Guided by these design insights, we propose DITRON, a hierarchical distributed tensor compiler. DITRON is structured into three layers: front-end, mid-end, and back-end. At the front-end, DITRON implements three hierarchical tiling levels: core-level tiling, device-level tiling, and task-level tiling. Core-level tiling enables users to leverage small static-shaped vectors or matrices to invoke hardware acceleration units such as Tensor Memory Accelerators (TMA) and Tensor Cores while maintaining full functional and performance compatibility with the existing Triton programming language. Device-level tiling unlocks the potential of hardware DMA/RDMA engines, which are optimized for high-throughput large-chunk data transfers. This tiling level adopts dynamic shapes and supports runtime shape computation, a critical capability for dynamic model architectures like Mixture-of-Experts (MoE). Task-level tiling goes a step further by enabling model-level tiling: it fuses the entire model into a single kernel, which is then deployed and executed across distributed clusters to maximize hardware resource utilization.

At the mid-end, DITRON translates operator-level collective communications into tile-level semantics and enables the overlapping of computation and communication. DITRON natively supports all major parallelism paradigms for LLM training and inference, including tensor parallelism, sequence parallelism, expert parallelism, and pipeline parallelism. Collective communication primitives such as AllGather, ReduceScatter, AllToAll, and AllReduce are dynamically partitioned into chunks, with each chunk mapped to a set of tiles from the corresponding computation kernels. Meanwhile, distributed synchronization events are embedded into computation kernels to enable seamless coordination with communication kernels.

At the back-end, DITRON offers a suite of hardware-agnostic primitives, encompassing address mapping, data access, and synchronization mechanisms. Being hardware-agnostic, these primitives can be translated into hardware-specific assembly code for any target backend by implementing corresponding translation rules. We have instantiated these translation rules for both NVIDIA and AMD backends, thereby enabling DITRON to support a broad spectrum of GPUs across these two platforms.

Through excessive validation and evaluation, DITRON is able to achieve $1.27 \times - 19.18 \times$ average speedup to vendor-provided non-overlapping kernels across a wide range of workloads, and is even 6% – 30% faster than expert-tuned overlapping libraries implemented in CUDA. It also renders 5% – 30% end-to-end performance improvement when integrated with vLLM (Kwon et al., 2023) for large batch size. On AMD GPUs the speedup to RocmBLAS+RCCL ranges from 1% – 38%; on PCIe GPUs the average speedup to CuBLAS+NCCL is $8.33 \times$.

2. Background and Related Work

To understand the design rationale behind DITRON, we first characterize the hardware hierarchy of modern distributed clusters and then analyze how existing programming models fail to align with this hierarchy.

2.1. The Hierarchy of Distributed Hardware

The memory and compute hierarchy of a distributed GPU cluster is inherently non-uniform. As illustrated in Figure 1, effective distributed programming requires managing data movement across three distinct domains, each with vastly different bandwidths and latencies.

For in-device domain (core-level), it involves data movement between HBM, L2 Cache, and registers. Optimization here relies on maximizing data reuse in SRAM and leveraging specialized compute units like Tensor Cores. Programs for this domain are often composed of instructions for grid-level, block-level, and warp-level computation and

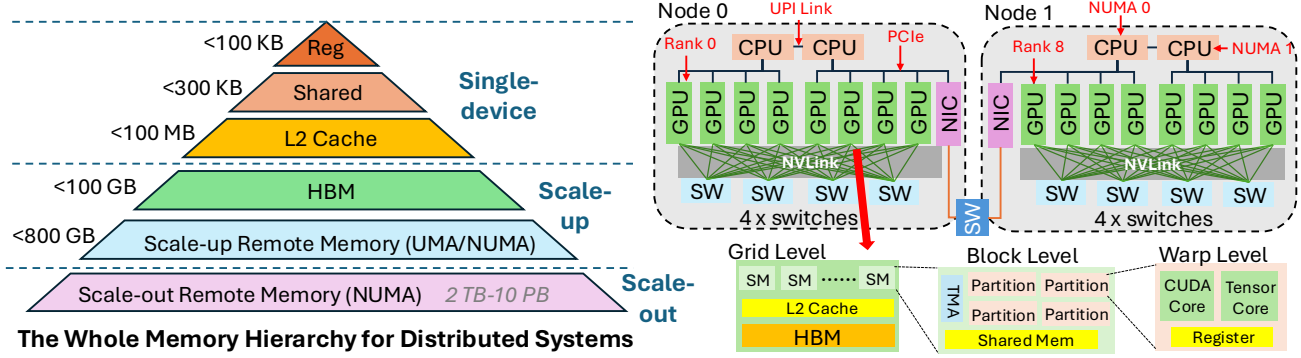


Figure 1. GPU Distributed Memory Hierarchy and Cluster Overview.

communication. For scale-up domain (intra-node), modern GPU (NVIDIA, 2023) nodes connect GPUs via high-bandwidth fabrics like NVLink. This domain supports Unified Memory Access (UMA) via load/store instructions, but also introduces NUMA characteristics where remote access latency is non-negligible. Crucially, this domain offers specialized hardware features such as NVLink Sharp (NVIDIA, 2018) for in-network reduction, which standard software often overlooks. For scale-out domain (inter-node), communication between nodes relies on Ethernet or InfiniBand. This domain is strictly NUMA, where data transfer requires explicit coordination via direct memory access (DMA) engines and network interface cards (NICs). The bandwidth gap here is significant: while HBM offers several TB/s bandwidth, inter-node links often operate at tens of GB/s bandwidth.

The fundamental challenge in distributed LLM workloads is that these levels are not isolated; a single operation (e.g., a distributed matrix multiplication) often spans all three domains simultaneously.

2.2. Limitations of Existing Programming Models

Despite the hierarchical nature of hardware, existing software stacks largely fail to provide a unified abstraction that captures these nuances.

Hand-tuned Libraries: Previous work (Chang et al., 2024; Zhang et al., 2025; Zhao et al., 2025; Gond et al., 2025) attempts to bridge the gap by providing hand-tuned kernels for computation-communication overlapping. For instance, libraries like FLUX (Chang et al., 2024) and COMET (Zhang et al., 2025) integrate communication primitives directly into CUTLASS kernels. However, their programming interfaces are often obscure and cumbersome to use, requiring deep knowledge of assembly (PTX) or complex C++ template metaprogramming. This rigidity makes it difficult for researchers to adapt these kernels to new model architectures.

Compilers and DSLs: Compilers such as CoCoNet (Jangda et al., 2022) and DistEinsum (Wang et al., 2023) provide operator-level DSLs for distributed programming. Recently, frameworks like Pallas (Google, 2025), TileLink (Zheng et al., 2025) and IRIS (Awad et al., 2025) have attempted to extend tiling concepts to distributed settings. While promising, they often struggle with the complexity of multi-level hierarchies. For example, TileLink primarily focuses on the scale-up abstraction but fails to expose the controls necessary for efficient scale-out communication.

3. DITRON System Design

Followed by the insights discussed in Section 1, we present DITRON, a unified compiler stack designed to bridge the gap between high-level distributed algorithms and low-level hardware heterogeneity. As illustrated in Figure 2, DITRON adopts a modular design comprising a hierarchical front-end interface, an optimization-centric mid-end swizzle, and a portable back-end code generation with unified primitives.

3.1. Front-end Three-level Programming Interface

The core innovation of DITRON lies in decoupling the logical view of a distributed tensor program from its physical execution. We introduce a three-level tiling abstraction that maps distinct program semantics to the corresponding hardware domains.

Core-level Interface: At the finest granularity, DITRON inherits the tile-level semantics from Triton to manage computation within a single GPU. This level operates on static-shaped tiles (e.g., 128×128 blocks) to maximize the utilization of specialized compute units like Tensor Cores and TMA engines. By maintaining compatibility with standard Triton syntax, DITRON allows users to seamlessly reuse existing optimized kernels for computation logic. We show the code to implement a high-performance AllGather+GEMM, GEMM+ReduceScatter, and GEMM+AllReduce overlapping kernels in Appendix D, where the computation kernels

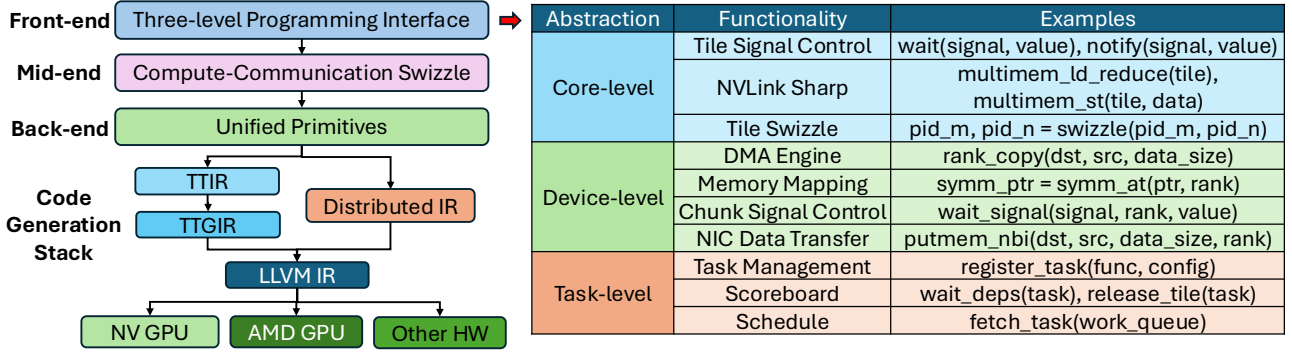


Figure 2. DITRON is composed of front-end interface, mid-end swizzle, and back-end primitives and code generation.

only require several lines of code modification (highlighted) to work for distributed systems. Core-level interface mainly contains signal control (*wait*, *notify*), on-switch computation (*multimem*), and tile swizzling as illustrated in the Table in Figure 2.

Device-level Interface: Unlike single-device compilers, DITRON introduces a device-level abstraction to manage data movement across the distributed memory hierarchy. This level operates on chunks, which are coarse-grained data blocks composed of multiple fine-grained tiles. This abstraction is designed with two key features. First, we use DMA-centric semantics. Unlike core-level load/store instructions, device-level primitives (e.g., *putmem*, *getmem*) map directly to asynchronous DMA engines and NICs, bypassing the GPU SMs to save compute resources. Second, we support dynamic shapes. To support dynamic model architectures like MoE, we allow the data chunk size, input size, and output size to be dynamically calculated at runtime. For instance, in an AllToAll operation, the size of chunks transferred between ranks can be determined dynamically based on token routing results.

Task-level Interface: At the highest level, DITRON treats the entire distributed workload as a directed acyclic graph (DAG) of tasks. This abstraction allows the compiler to fuse communication and computation kernels into a single MegaKernel. By managing task dependencies and scheduling globally, DITRON eliminates the overhead of repeated kernel launches and enables persistent residency of kernels on the hardware, ensuring that communication fabrics and compute units are kept constantly busy. Different from previous MegaKernel work (Wu et al., 2025; Spector et al., 2025) that requires CUDA or C++ programming, DITRON allows users to register native Triton kernels as tasks and then automatically schedules these kernels at compile time through software-maintained scoreboard. Finally, these Triton kernels are converted to a fused MegaKernel. We show how to register Triton kernels as tasks and the final generated MegaKernel in Appendix D.4.

3.2. Mid-end Compute-Communication Swizzling

The Mid-end is responsible for lowering high-level compute-communication algorithm into overlapped versions and applying system-aware optimizations. The most critical optimization in DITRON is distributed swizzling.

In a distributed settings, the latency of accessing remote memory is orders of magnitude higher than local HBM. Standard sequential execution often leaves compute units idle while waiting for data. DITRON addresses this by re-ordering the execution of tiles. We formalize this into two distinct modes handled by the compiler:

Gather Mode: For operations where computation depends on remote data (e.g., AllGather+GEMM), the compiler schedules remote data fetch requests as early as possible, effectively treating local HBM as a cache for remote memory.

Scatter Mode: For operations where results must be sent out (e.g., GEMM+ReduceScatter), the compiler prioritizes the computation of tiles destined for the furthest remote nodes, ensuring that long-haul communication is initiated immediately upon data availability.

The swizzling logic is encapsulated within the compiler as a stateless, JIT-compatible utility. Leveraging Triton’s JIT infrastructure and DITRON’s three-level interfaces, we expose swizzling as a pluggable primitive that can be flexibly injected into any kernel. The transformation is defined as:

```
new_pid = swizzle_func(old_pid,
                        shape_to_swizzle, rank, world_size, block_size)
```

We provide the detailed implementation in Appendix A. To illustrate its efficacy, Figure 3 visualizes the execution flow of a GEMM+ReduceScatter workload. In a naive schedule (without swizzling), all ranks sequentially process tiles starting from index 0. Since the data required for tile 0 typically resides on Rank 0, all other ranks suffer from dependency

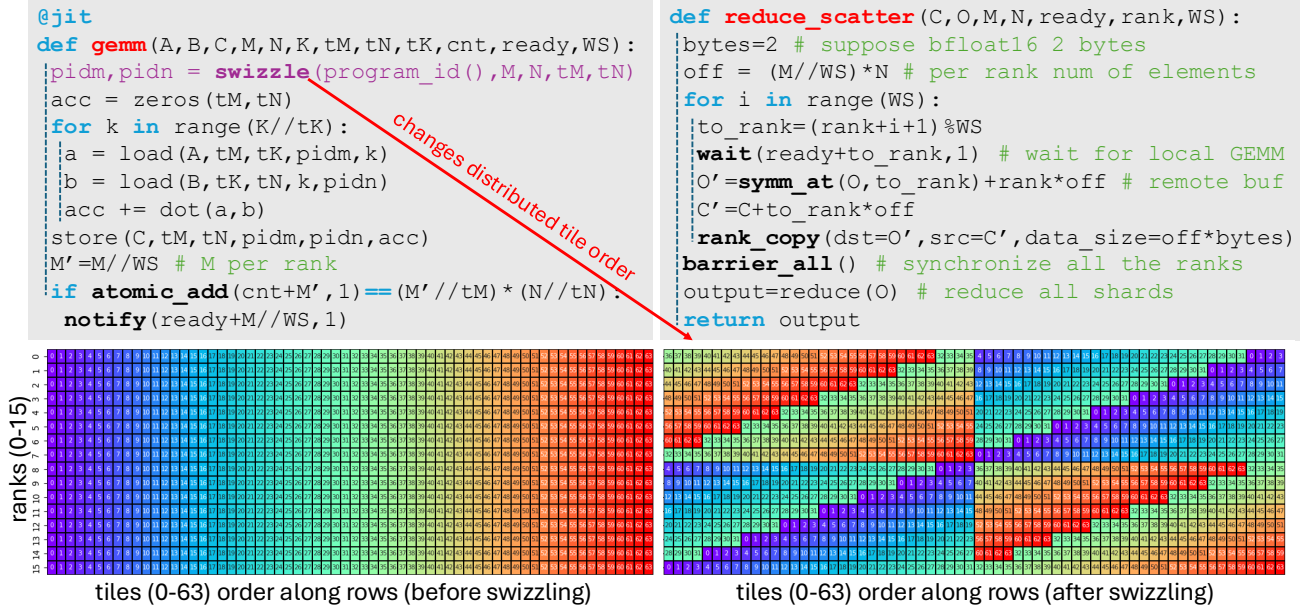


Figure 3. Simplified GEMM+ReduceScatter code example and distributed swizzling. The ReduceScatter requires that each GPU rank i takes 4 data chunks ($[4i, 4i + 4)$) eventually. Tiles are processed from left to right. Without swizzling, all ranks start from data tiles for rank 0 and the other ranks are blocked, while with swizzling, all the ranks can start without blocking. A clearer enlarged swizzle view is presented in the Appendix A and the full code example is in Appendix D.2.

stalls while waiting for data transfer. Conversely, DITRON applies a rank-aware offset, which specifically starts execution at $rank_id \bmod local_world_size$. This ensures that each rank prioritizes tiles for which data is locally available or already arriving, thereby effectively overlapping the GEMM computation with the ReduceScatter communication. Swizzling for imperfect shapes can be harder and we place the discussion in Appendix A.

3.3. Back-end Primitives and Code Generation

Primitives: To achieve hardware portability (design principle 3), DITRON abstracts backend-specific communication functionalities into a set of hardware-agnostic primitives compliant with the OpenSHMEM standard. During the code generation phase, DITRON’s Distributed IR is lowered into LLVM IR. We utilize LLVM’s CallExtern capability to link against vendor-specific communication libraries.

The primitives are divided into three classes: distributed primitives, SIMT primitives, and SHMEM device primitives. Distributed primitives are used to generate low-level code for distributed signal control. SIMT primitives are used for threads cooperation or synchronization. SHMEM device primitives are used for remote data transfer and signal exchange. In Appendix E we list these primitives in detail.

Compiler Stack: DITRON is implemented with around 59,000 lines of Python code and 7,000 lines of C++ code,

consisting primarily of a compiler stack and an optimized kernel library.

The compiler stack of DITRON is illustrated in Figure 2. Users write Triton-like programs using our three levels of tiling interfaces, which cover both computation and communication. The hardware primitives are encapsulated in our Distributed IR for compilation, while standard single-device semantics (e.g., *dot*, *load/store*) in the programs are lowered to standard Triton IR (TTIR) and Triton GPU IR (TTGIR). This Distributed IR with OpenSHMEM semantic is then lowered to assembly with extern symbol linked to hardware-specific low-level library such as NVSHMEM (for NVIDIA) and rocSHMEM (for AMD). To support other hardware, we can also leverage custom SHMEM libraries provided by vendors. DITRON currently support more than five different types of GPUs and different communication fabrics including NVLink, xGMI, PCIe, and IB.

Optimized Kernel Library: Beside compiler stack, we also provide a comprehensive kernel library in DITRON, which are implemented using the three-level tiling interface provided by DITRON. These kernels have been validated in industry LLM training and inference for both numeric precision and performance. We list them in Table 1.

To achieve the best performance, DITRON incorporates several low-level optimizations to address specific hardware constraints and latency bottlenecks in distributed systems. And we pinpoint three of them:

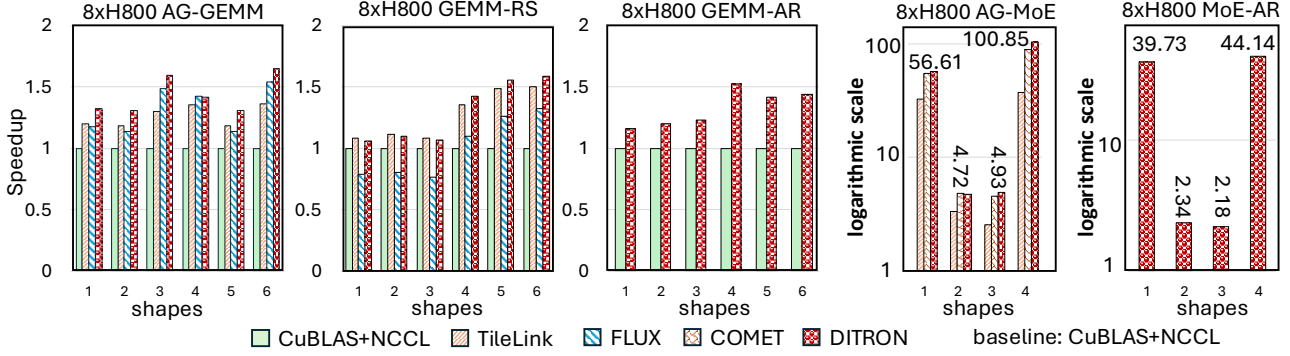


Figure 4. Evaluation of single workload on $8 \times$ H800 GPUs. Workloads are AllGather+GEMM (AG-GEMM), GEMM+ReduceScatter (GEMM-RS), GEMM+AllReduce (GEMM-AR), AllGather+MoE (AG-MoE), MoE+AllReduce (MoE-AR). A logarithmic scale is used for the speedup of MoE due to its wide span, and the specific values are listed in the Appendix C.

Table 1. The Optimized Kernel List in DITRON

Kernel Name	Brief Description
ag_gemm	Fused AllGather + GEMM
ag_group_gemm	Fused AllGather + Group GEMM
fast_allgather	Low-latency AllGather
gemm_rs	GEMM + ReduceScatter
gemm_allreduce	Fused GEMM + AllReduce
flash_decode_gqa	GQA Batch Flash Decode
sp_ag_attn	Seq Parallel AllGather Attn
fast_all_to_all	Optimized All-to-All
a2a_single_2d	2D Single Kernel All-to-All
a2a_single_gemm	Fused All-to-All + GEMM
ulysses_comm	Ulysses SP Pre-Attn Comm

- **Low-Latency (LL) Protocol Integration.** Standard communication protocols often prioritize bandwidth at the cost of synchronization latency. Following the design of NCCL (NVIDIA, 2024), DITRON implements a specialized Low-Latency (LL) protocol that bypasses expensive synchronization handshakes.
- **Device-to-Device (D2D) Copy Fusion.** We observe that discrete data movement calls via driver APIs (e.g., `cudaMemcpy`) or framework ops (e.g., `torch.copy`) often introduce launch jitters and non-deterministic SM utilization, creating stragglers that negate the benefits of overlapping. To mitigate this, DITRON fuses these D2D copy operations directly into the generated communication or computation kernels. This fusion eliminates driver overhead and ensures deterministic resource scheduling.
- **PCIe-Aware Synchronization and Topology.** Commodity PCIe interconnects lack hardware guarantees for atomic memory ordering. To deploy DITRON on PCIe-based GPUs, we implement software barriers using `volatile` load/store instructions to enforce memory consistency without hardware atomics.

4. Evaluation

We evaluate DITRON across diverse parallel configurations. For inference (intra-node TP), we benchmark 5 GEMM/MoE workloads, Attention/FFN modules, and end-to-end Qwen3-32B/LLaMA3-70B models. For training, we analyze weak and strong scaling of TP, SP, and EP kernels on 8–128 GPUs. Finally, we demonstrate support for AMD and PCIe GPUs.

4.1. Evaluation for Inference

Single Workload Evaluation: We evaluated 5 distinct workloads on $8 \times$ H800 GPUs across 26 configurations, with each workload featuring multiple shape configurations derived from real-world models such as LLaMA (Dubey et al., 2024), Mixtral (Jiang et al., 2024), GPT (OpenAI, 2023), Qwen (Yang et al., 2025), and DeepSeek (DeepSeek-AI et al., 2024a). Detailed shape configurations are provided in Appendix B.2. Our baselines include CuBLAS (NVIDIA, 2022)+NCCL (NVIDIA, 2024) (non-overlapping), TileLink (Zheng et al., 2025), FLUX (Chang et al., 2024), and COMET (Zhang et al., 2025), with results presented in Figure 4.

Overall, for AG-GEMM, the geometric speedup of DITRON is $1.43\times$ to CuBLAS+NCCL, $1.13\times$ to TileLink, and $1.09\times$ to FLUX. For GEMM-RS, the geometric speedup of DITRON is $1.27\times$ to CuBLAS+NCCL, $1.02\times$ to TileLink, and $1.30\times$ to FLUX. TileLink and FLUX have no support for AllReduce. For GEMM-AR, the geometric speedup to CuBLAS+NCCL is $1.32\times$. The speedup of AG-GEMM and GEMM-RS mainly comes from the overlapping of communication instead of faster GEMM (actually, we use Triton’s GEMM and the GEMM is slightly slower than that of CuBLAS and FLUX), nearly 87.5% communication latency is hidden by computation for large input shapes. The speedup of GEMM-AR comes from faster AllReduce implemented using DITRON, which sup-

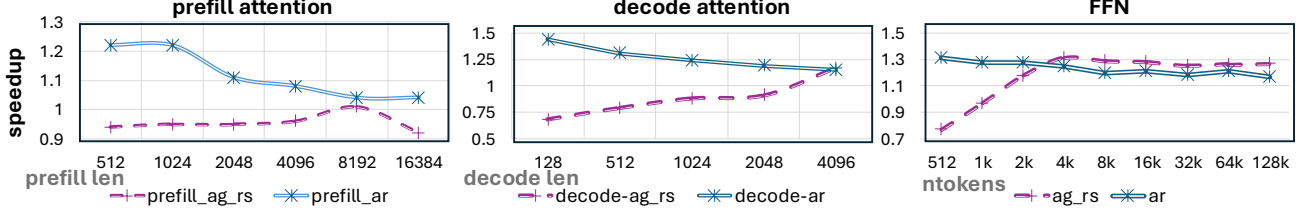


Figure 5. Module-level TP evaluation for Qwen3-32B: Results for prefill and decode attention show that DITRON with AllReduce achieves better speedups over CuBLAS+NCCL compared to using AllGather and ReduceScatter. For MLP, results indicate that the combination of AllGather and ReduceScatter performs better for large shapes, while AllReduce is superior for small shapes.

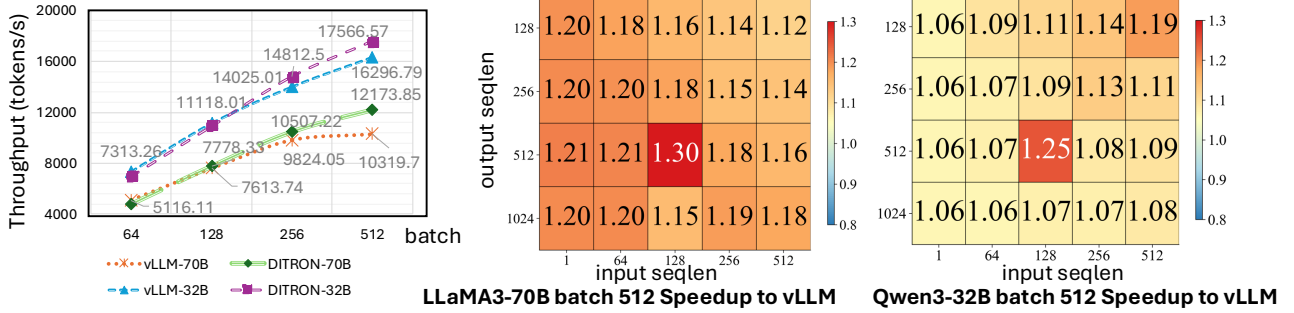


Figure 6. End-to-end TP evaluation results for Qwen3-32B and LLaMA3-70B on $8 \times$ H800s: DITRON demonstrates advantages over vLLM for large batch sizes and large models.

ports both one-shot algorithm and two-shot algorithm via multi-memory reduction/broadcast provided by NVLink Sharp. For AG-MoE, the geometric speedup of DITRON is $19.18\times$ to CuBLAS+NCCL, $1.89\times$ to TileLink, and $1.06\times$ to COMET. For MoE-AR, the average speedup to CuBLAS+NCCL is $13.89\times$.

Module-level Evaluation: We incorporate the aforementioned workloads into attention modules and FFN modules. The QKV projection and output projection in attention modules are replaced by DITRON kernels. Prefill and decode performance is presented in Figure 5. Prefill input lengths range from 512 to 16k, and decode output lengths range from 128 to 4k. The two lines represent the difference in speedups over CuBLAS+NCCL between using AllGather+ReduceScatter and using AllReduce. The results indicate that for attention modules, both prefill and decode prefer AllReduce communication because the reduction dimension for GEMM (head dimension size in attention) is small (128), making GEMM latency not enough to hide communication latency of AllGather or ReduceScatter. The geometric mean speedup of DITRON’s attention module (using AllReduce) to CuBLAS+NCCL is $1.12\times$ for prefill and $1.26\times$ for decode. On the other hand, GEMMs in FFN modules use large intermediate size and the latency of GEMMs can hide communication latency when given enough input tokens (large batch or sequence length), so overlapping AllGather and ReduceScatter is better than using AllReduce

for sequence length larger than 2k. The speedup for 128k tokens of DITRON is $1.17\times$ using AllReduce and $1.27\times$ using AllGather+ReduceScatter.

End-to-end Evaluation: We evaluate DITRON using end-to-end models LLaMA3-70B (Dubey et al., 2024) and Qwen3-32B (Yang et al., 2025). We integrate DITRON into vLLM (Kwon et al., 2023) and compare the performance with/without DITRON. The results are shown in Figure 6. vLLM (Kwon et al., 2023) without DITRON is still a strong baseline as vLLM natively employs efficient AllReduce kernels designed by experts. For batch size less than 128, vLLM is slightly better than DITRON in throughput. But for batch sizes larger than 128, DITRON achieves 5% – 30% speedup to vLLM. Specially, for batch size 512, we show the detailed speedup under different input lengths and output lengths for LLaMA3-70B and Qwen3-32B in Figure 6. The speedup translates to 12k tokens/s throughput for LLaMA3-70B and 17k tokens/s throughput for Qwen3-32B.

Distributed MegaKernel Evaluation: For single batch inference, the hardware resources are usually underutilized, task-level scheduling that produces MegaKernel can eliminate kernel launch overhead, increase SM activity, and improve end-to-end performance. We compare DITRON’s distributed MegaKernel with PyTorch (Eager mode and CUDAGraph mode), DITRON+CUDAGraph, and Mirage (Wu et al., 2025) as shown in Figure 8. The geometric speedup is $6.28\times$ to Torch Eager, $1.73\times$ to Mirage, $1.33\times$ to

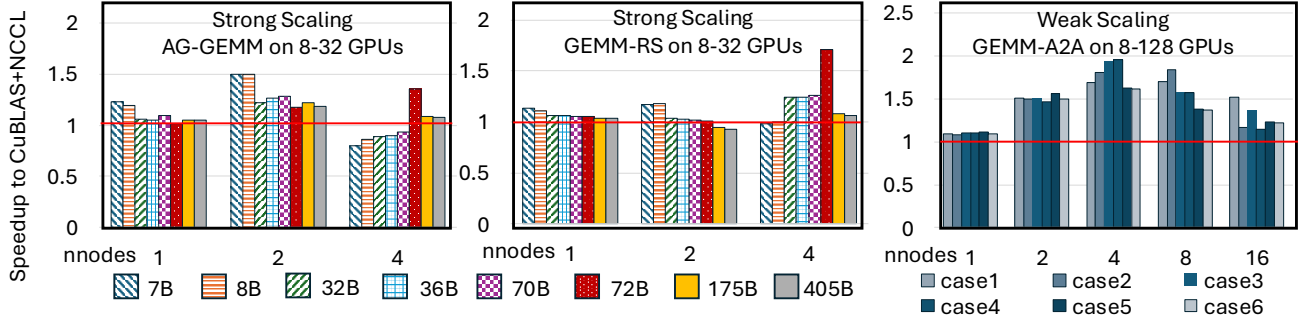


Figure 7. Scaling TP and SP workloads on Hopper (96GB HBM) GPU clusters: Results are presented as speedups over CuBLAS+NCCL, with shapes derived from various real-world LLMs (details in Appendix B.2). DITRON maintains speedups for AG-GEMM across up to 16 GPUs. For 32 GPUs, speedups are achievable only with shapes from large LLMs. The speedup of GEMM-RS remains consistent across 8–32 GPUs. GEMM-A2A exhibits the best weak scaling due to its scalable AllToAll.

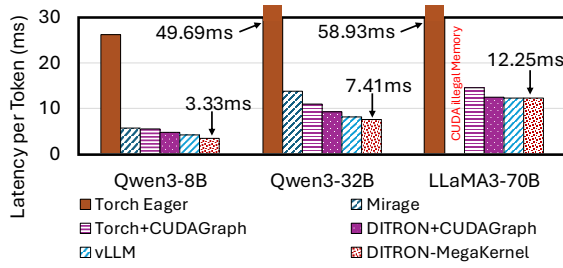


Figure 8. Inference latency for Qwen3-8B, Qwen3-32B, LLaMA-70B using DITRON’s distributed Megakernel on 8× H800.

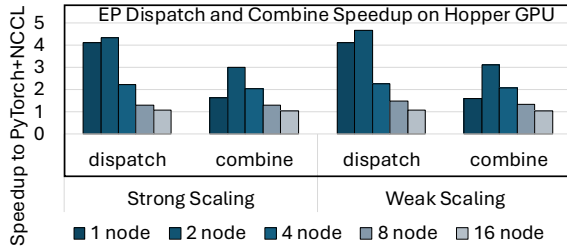


Figure 9. Scaling EP dispatch and combine using DITRON.

Torch+CuDAGraph, 1.11× to DITRON+CuDAGraph, and 1.10× to vLLM. We also add MegaKernel code examples in Appendix D.4.

4.2. Evaluation for Training

For training, we evaluate both strong scaling and weak scaling performance for TP (AllGather+GEMM, GEMM+ReduceScatter), SP (GEMM+AllToAll), and EP (MoE Dispatch and MoE Combine) using DITRON. The results for TP and SP are shown in Figure 7 and the results for EP are shown in Figure 9. The detailed shape configurations are put in Appendix B.2. More results (on different GPUs) are put in Appendix C.

TP workloads renders heavy communication among nodes

and the low bandwidth of network communication makes scaling non-beneficial. As a result, we only observe speedups of AllGather+GEMM and GEMM+ReduceScatter for 8-32 GPUs (strong scaling, number of total tokens is 32768), which ranges from 0.80× to 1.71× to CuBLAS+NCCL. For cases where the speedup is lower than 1, the main cause is GEMM becomes too small for each GPU after sharding, which cannot hide communication latency. As a comparison, GEMM+AllToAll (GEMM-A2A) gives consistent speedups for weak scaling (sequence length per rank remains unchanged) from 8 GPUs to 128 GPUs, where the shape of GEMM keeps constant. As for EP (we set 8192 tokens per rank, topk 8, hidden size 7168), dispatch and combine performance remains similar for strong scaling (globally 512 experts in total) and weak scaling (8 experts per rank). The speedup to PyTorch+NCCL implementations ranges from 1.04× to 4.70×.

4.3. Ablation study

We provide a detailed breakdown using an end-to-end Llama example (8×GPUs), isolating the contributions of overlapping, swizzling, and tiling optimizations. We test two settings for TP inference: overlap using AllReduce (AR) and overlap using AllGather/ReduceScatter (AG+RS).

Table 2. Ablation study on 8×H800 Llama inference.

Configuration	Tok/s	Δ
<i>GEMM + AllReduce</i>		
Full (DITRON)	12,174	—
w/o cross-rank swizzle	11,605	−4.7%
w/o multi-level tiling	11,706	−3.8%
w/o overlap (vLLM baseline)	10,320	−15.2%
<i>AllGather + GEMM / GEMM + ReduceScatter</i>		
Full (DITRON)	10,928	—
w/o cross-rank swizzle	10,642	−2.6%
w/o multi-level tiling	9,494	−13.1%
w/o overlap (vLLM baseline)	10,320	−5.6%

For GEMM+AllReduce, cross-rank swizzling, tiling, and overlapping contribute 4.7%, 3.8%, and 15.2% respectively. For AllGather+GEMM and GEMM+ReduceScatter, the contributions are 2.6%, 13.1%, and 5.6%.

4.4. Support for Other Platforms

DITRON can be transferred to more hardware platforms due to the flexibility of our distributed IR and the compatibility of dependent Triton compiler. For now, we manage to support AMD GPUs and PCIe GPUs. We put our preliminary results in Appendix C. Overall, on AMD GPUs the speedup to RocmBLAS+RCCL ranges from 2% – 38%; on PCIe GPUs the average speedup to CuBLAS+NCCL is $8.33\times$.

4.5. Support for Emerging Operators

DITRON makes it easier to prototype high-performance overlapping implementations for novel algorithms. To demonstrate this, we use DITRON to implement SP Gated Linear Attention (GLA) kernels, and compare with the open-sourced SOTA baseline (LASP2 (Sun et al., 2025)) on $8\times$ Hopper GPUs. The performance gain is achieved by allowing fully customized communication to overlap with computation, forming a tile-level communication-computation pipeline.

Table 3. GLA SP kernel performance (32K tokens/rank).

Setting	Baseline [†]	DITRON
16 GPUs	6.64 ms	6.18 ms
64 GPUs	8.39 ms	6.48 ms
128 GPUs	11.07 ms	6.83 ms

[†]LASP2 (Sun et al., 2025), adopted by flash-linear-attention (Yang & Zhang, 2024).

4.6. Enterprise-level Deployment

Hybrid parallelism support: DITRON fully supports hybrid parallelism, as different types of distributed kernels can be composed freely. Technically, we initialize a single global NVSHMEM team and split it into sub-teams (PP, SP, EP), each running their respective kernels within the corresponding sub-team scope. This team-split mechanism naturally supports arbitrary hybrid parallelism configurations.

Training Integration. We have adapted the training of MoE models of various sizes. In tasks ranging from models with several billion parameters to hundreds of billions of parameters, different layers are accelerated by DITRON. For the Attention module, we employ SP Attention. We overlap the GEMM and AllToAll communication within Attention. Compared with the original Megatron (Narayanan et al., 2021) implementation, the per-module speedup ex-

ceeds 20%. For the MoE module, we overlap Dispatch, GroupedGEMM, and Combine operations, achieving a per-module speedup of more than $2\times$. Compared with the native Megatron implementation, the end-to-end gain reaches 10%. Even compared with the highly tuned handwritten CUDA overlap implementation (FLUX (Chang et al., 2024)), we achieve equivalent performance, while reducing the code length by more than an order of magnitude and cutting the development cycle from months to days. For the Optimizer, we support optimizations for the Muon Optimizer (Liu et al., 2025), delivering a per-module speedup of over 20%. For Pipeline Parallelism, we implement an efficient PP communication kernel. It can saturate the bandwidth using only 8 SMs within a node, and only 1 SM across nodes, while supporting flexible overlap with other layers. All our adapted kernels are bitwise identical to the native implementations, fully ensuring the accuracy and stability of training.

Inference Integration. For inference services, we deploy high-performance TP inference services in the cloud. We accelerate TP inference by overlapping and fusing AllReduce and GEMM operations, supporting various inference scenarios including PCIe and NVLink GPUs, with an end-to-end performance speedup of approximately 20%. For edge inference, such as inference on robotic devices, we provide high-performance TP Attention and TP GEMM implementations, achieving an end-to-end speedup of over 20%.

5. Conclusion

Distributed inference and training become necessary and require more researchers to be able to program distributed kernels. This work presents DITRON, a flexible and versatile distributed compiler with core-level, device-level, and task-level interfaces for overlapping kernels. Researchers can use DITRON to program efficient distributed kernels for different parallelism with performance comparable to or better than expert-tuned kernels.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- AMD. URL <https://github.com/ROCm/rocSHMEM>.
- Anthropic. Claude 3.5 sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>, 2024.

- Awad, M., Osama, M., and Potter, B. Iris: First-class multi-GPU programming experience in Triton, 2025.
- Chang, L., Bao, W., Hou, Q., Jiang, C., Zheng, N., Zhong, Y., Zhang, X., Song, Z., Jiang, Z., Lin, H., Jin, X., and Liu, X. FLUX: fast software-based communication overlap on gpus through kernel fusion. *CoRR*, abs/2406.06858, 2024. doi: 10.48550/ARXIV.2406.06858. URL <https://doi.org/10.48550/arXiv.2406.06858>.
- Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., Blistein, M., Ram, O., Zhang, D., Rosen, E., et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Deng, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Yang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Chen, J., Yuan, J., Qiu, J., Song, J., Dong, K., Gao, K., Guan, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Pan, R., Xu, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Zheng, S., Wang, T., Pei, T., Yuan, T., Sun, T., Xiao, W. L., Zeng, W., An, W., Liu, W., Liang, W., Gao, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Chen, X., Nie, X., and Sun, X. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *CoRR*, abs/2405.04434, 2024a. doi: 10.48550/ARXIV.2405.04434. URL <https://doi.org/10.48550/arXiv.2405.04434>.
- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T., Xiao, W. L., and Zeng, W. Deepseek-v3 technical report. *CoRR*, abs/2412.19437, 2024b. doi: 10.48550/ARXIV.2412.19437. URL <https://doi.org/10.48550/arXiv.2412.19437>.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Rozière, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I. M., Misra, I., Evtimov, I., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billorey, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., and et al. The llama 3 herd of models. *CoRR*, abs/2407.21783, 2024. doi: 10.48550/ARXIV.2407.21783. URL <https://doi.org/10.48550/arXiv.2407.21783>.
- Gond, R., Kwatra, N., and Ramjee, R. Tokenweave: Efficient compute-communication overlap for distributed llm inference, 2025. URL <https://arxiv.org/abs/2505.11329>.
- Google. Pallas, 2025. URL <https://docs.jax.dev/en/latest/pallas/index.html>.
- Hwang, C., Cheng, P., Dathathri, R., Jangda, A., Maleki, S., Musuvathi, M., Saarikivi, O., Shah, A., Yang, Z., Li, B., Rocha, C., Zhou, Q., Ghazimirsaeed, M., Anantharamu, S., and Jose, J. Msccl++: Rethinking gpu communication abstractions for ai inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '26*, pp. 1201–1215, New York, NY, USA, 2026. Association for Computing Machinery. ISBN 9798400723599. doi: 10.1145/3779212.3790188. URL <https://doi.org/10.1145/3779212.3790188>.
- Jangda, A., Huang, J., Liu, G., Sabet, A. H. N., Maleki, S., Miao, Y., Musuvathi, M., Mytkowicz, T., and Saarikivi, O. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Falsafi, B., Ferdman, M., Lu, S., and Wenisch, T. F. (eds.), ASPLOS '22: 27th ACM International Conference on*

- Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, 28 February 2022 - 4 March 2022, pp. 402–416. ACM, 2022. doi: 10.1145/3503222.3507778. URL <https://doi.org/10.1145/3503222.3507778>.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de Las Casas, D., Hanna, E. B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud, L. R., Saulnier, L., Lachaux, M., Stock, P., Subramanian, S., Yang, S., Antoniak, S., Scao, T. L., Gervet, T., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mixtral of experts. *CoRR*, abs/2401.04088, 2024. doi: 10.48550/ARXIV.2401.04088. URL <https://doi.org/10.48550/arXiv.2401.04088>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In Flinn, J., Seltzer, M. I., Druschel, P., Kaufmann, A., and Mace, J. (eds.), *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Liao, G., Qin, H., Wang, Y., Golden, A., Kuchnik, M., Yetim, Y., Ang, J. J., Fu, C., He, Y., Hsia, S., Jiang, Z., Li, D., Pashkevich, U., Puvvada, V., Shi, F., Steiner, M., Xiao, R., Yan, N., Yu, X., Fang, Z., Zainul-Abedin, A., Singh, K., Yu, H., Chi, W., Huang, B., Zhang, S., Weller, N., Marine, Z., Cook, W., Wu, C.-J., and Liu, G. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta, 2025. URL <https://arxiv.org/abs/2512.23236>.
- Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., Chen, Y., Zheng, H., Liu, Y., Liu, S., Yin, B., He, W., Zhu, H., Wang, Y., Wang, J., Dong, M., Zhang, Z., Kang, Y., Zhang, H., Xu, X., Zhang, Y., Wu, Y., Zhou, X., and Yang, Z. Muon is scalable for LLM training. *CoRR*, abs/2502.16982, 2025. doi: 10.48550/ARXIV.2502.16982. URL <https://doi.org/10.48550/arXiv.2502.16982>.
- Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., Phanishayee, A., and Zaharia, M. Efficient large-scale language model training on GPU clusters using megatron-lm. In de Supinski, B. R., Hall, M. W., and Gamblin, T. (eds.), *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2021, St. Louis, Missouri, USA, November 14-19, 2021*, pp. 58. ACM, 2021. doi: 10.1145/3458817.3476209. URL <https://doi.org/10.1145/3458817.3476209>.
- NVIDIA. Nvidia nvswitch: Technical overview. Technical report, NVIDIA, 2018. URL <https://images.nvidia.com/content/pdf/nvswitch-technical-overview.pdf>.
- NVIDIA. cuBLAS, 2022. URL <https://developer.nvidia.com/cublas>.
- NVIDIA. Hopper architecture whitepaper. Technical report, NVIDIA, 2023. URL <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>.
- NVIDIA. Nvidia collective communications library. <https://developer.nvidia.com/nccl>, 2024.
- OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023. doi: 10.48550/ARXIV.2303.08774. URL <https://doi.org/10.48550/arXiv.2303.08774>.
- Qwen-Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Rivière, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., Tsitsulin, A., Vieillard, N., Stanczyk, P., Girgin, S., Momchev, N., Hoffman, M., Thakoor, S., Grill, J., Neyshabur, B., Bachem, O., Walton, A., Severyn, A., Parrish, A., Ahmad, A., Hutchison, A., Abdagic, A., Carl, A., Shen, A., Brock, A., Coenen, A., Laforge, A., Paterson, A., Bastian, B., Piot, B., Wu, B., Royal, B., Chen, C., Kumar, C., Perry, C., Welty, C., Choquette-Choo, C. A., Sinopalnikov, D., Weinberger, D., Vijaykumar, D., Rogozinska, D., Herbison, D., Bandy, E., Wang, E., Noland, E., Moreira, E., Senter, E., Eltyshchev, E., Visin, F., Rasskin, G., Wei, G., Cameron, G., Martins, G., Hashemi, H., Klimczak-Plucinska, H., Batra, H., Dhand, H., Nardini, I., Mein, J., Zhou, J., Svensson, J., Stanway, J., Chan, J., Zhou, J. P., Carrasqueira, J., Iljazi, J., Becker, J., Fernandez, J., van Amersfoort, J., Gordon, J., Lipschultz, J., Newlan, J., Ji, J., Mohamed, K., Badola, K., Black, K., Millican, K., McDonnell, K., Nguyen, K., Sodhia, K., Greene, K., Sjöstrand, L. L., Usui, L., Sifre, L., Heuermann, L., Lago, L., and McNealus, L. Gemma 2: Improving open language models at a practical size. *CoRR*, abs/2408.00118, 2024. doi: 10.48550/ARXIV.2408.00118. URL <https://doi.org/10.48550/arXiv.2408.00118>.
- Spector, B., Juravsky, J., Sul, S., Dugan, O., Lim, D., Fu, D., Arora, S., and Ré, C. Look ma, no bubbles! designing a low-latency megakernel for LLAMA-1B. <https://hazyresearch.stanford.edu/>

- [blog/2025-05-27-no-bubbles](#), 2025. Hazy Research Blog.
- Sun, W., Lan, D., Zhong, Y., Qu, X., and Cheng, Y. LASP-2: rethinking sequence parallelism for linear attention and its hybrid. *CoRR*, abs/2502.07563, 2025. doi: 10.48550/ARXIV.2502.07563. URL <https://doi.org/10.48550/arXiv.2502.07563>.
- TileLang-Team. Tilelang, 2025. URL <https://github.com/tile-ai/tilelang>.
- Tillet, P., Kung, H., and Cox, D. D. Triton: an intermediate language and compiler for tiled neural network computations. In Mattson, T., Muzahid, A., and Solar-Lezama, A. (eds.), *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL@PLDI 2019, Phoenix, AZ, USA, June 22, 2019*, pp. 10–19. ACM, 2019. doi: 10.1145/3315508.3329973. URL <https://doi.org/10.1145/3315508.3329973>.
- Wang, S., Wei, J., Sabne, A., Davis, A., Ilbeyi, B., Hechtman, B., Chen, D., Murthy, K. S., Maggioni, M., Zhang, Q., Kumar, S., Guo, T., Xu, Y., and Zhou, Z. Overlap communication with dependent computation via decomposition in large deep learning models. In Aamodt, T. M., Jerger, N. D. E., and Swift, M. M. (eds.), *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, pp. 93–106. ACM, 2023. doi: 10.1145/3567955.3567959. URL <https://doi.org/10.1145/3567955.3567959>.
- Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. Mirage: A multi-level superoptimizer for tensor programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, Boston, MA, July 2025. USENIX Association.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL <https://doi.org/10.48550/arXiv.2505.09388>.
- Yang, S. and Zhang, Y. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, January 2024. URL <https://github.com/fla-org/flash-linear-attention>.
- Zhang, S., Zheng, N., Lin, H., Jiang, Z., Bao, W., Jiang, C., Hou, Q., Cui, W., Zheng, S., Chang, L., Chen, Q., and Liu, X. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. *CoRR*, abs/2502.19811, 2025. doi: 10.48550/ARXIV.2502.19811. URL <https://doi.org/10.48550/arXiv.2502.19811>.
- Zhao, C., Zhou, S., Zhang, L., Deng, C., Xu, Z., Liu, Y., Yu, K., Li, J., and Zhao, L. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>, 2025.
- Zheng, S., Fang, J., Zheng, X., Hou, Q., Bao, W., Zheng, N., Jiang, Z., Wang, D., Ye, J., Lin, H., Chang, L.-W., and Liu, X. Tilelink: Generating efficient compute-communication overlapping kernels using tile-centric primitives, 2025. URL <https://arxiv.org/abs/2503.20313>.

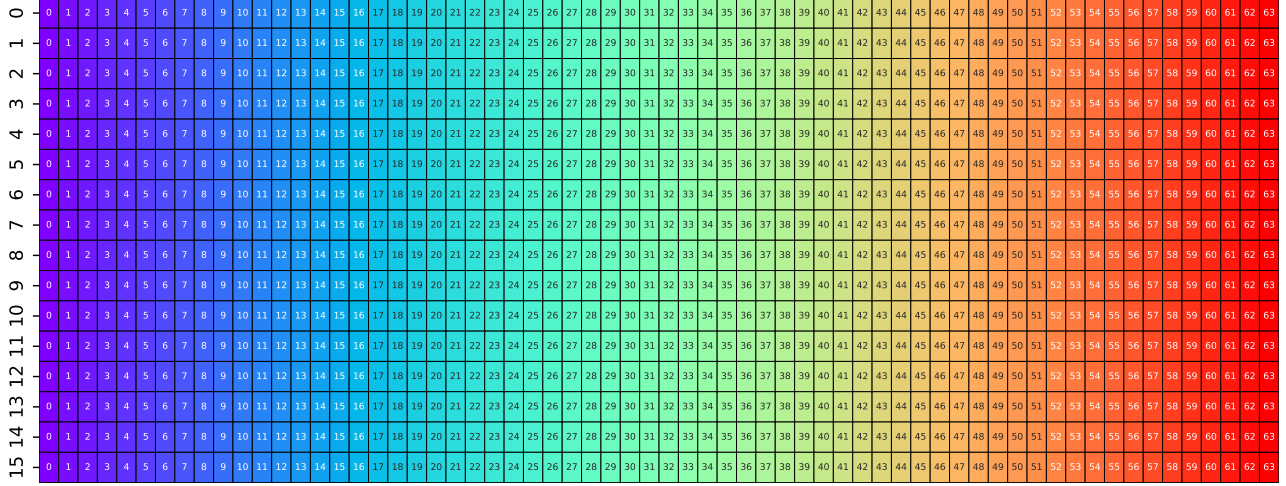


Figure 10. Global view of ranks and data tiles without swizzling. Corresponding to Figure 3. Each row corresponds to one rank; each column corresponds to one tile. Each rank computes GEMM from the left tile to the right tile. This example is for GEMM+ReduceScatter.

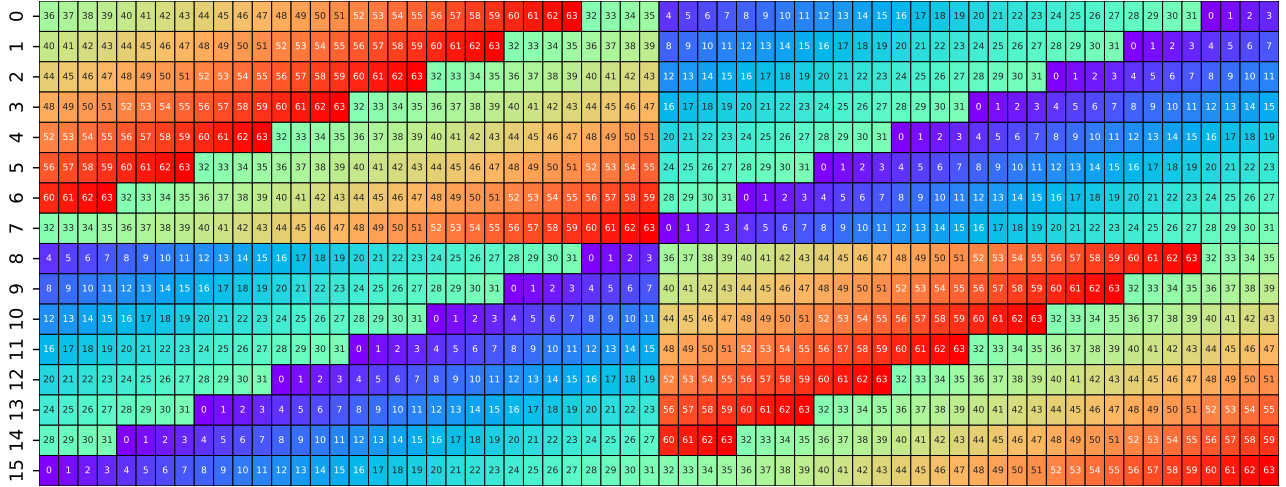


Figure 11. Global view of ranks and data tiles with swizzling. Corresponding to Figure 3. This example is for GEMM+ReduceScatter.

A. Swizzle

Swizzling optimization has been mentioned in previous work (Jangda et al., 2022; Chang et al., 2024; Zheng et al., 2025). But their swizzling methods are limited to single node and there lack a clear explanation of the swizzling for non-perfect tiling scenarios (which is the most common for LLM training and inference).

Figure 10 and Figure 11 show an enlarged view of Figure 3, which correspond to swizzle optimization for 16 GPUs GEMM+ReduceScatter with perfect shapes (no cross-rank tiles). For this case, we use $M_{per_rank} = 1024$, $N_{per_rank} = 256$, $tile_M = tile_N = 256$.

For non-perfect shapes, some tiles are needed by multiple ranks for subsequent ReduceScatter. These tiles should be permuted to the left so that they can be computed earlier and transferred to the ranks that need them. Figure 12 shows swizzle results for 8 GPUs; Figure 13 shows swizzling for 16 GPUs; and Figure 14 shows swizzling for 32 GPUs. For this case, we use $M_{per_rank} = 997$, $N_{per_rank} = 256$, $tile_M = tile_N = 256$.

Besides GEMM+ReduceScatter, we also implement similar swizzling for AllGather+GEMM, and AllGather+MoE. The

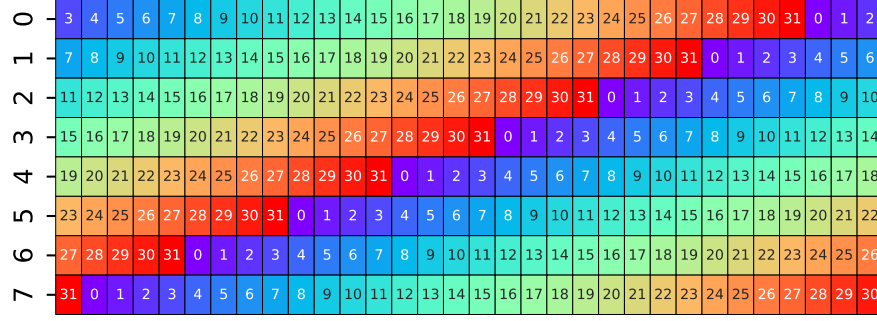


Figure 12. Single node global view of swizzling for non-perfect tiling. The tile orders are different from that of perfect tiling. Cross-rank tiles are permuted to left. This example is for GEMM+ReduceScatter.

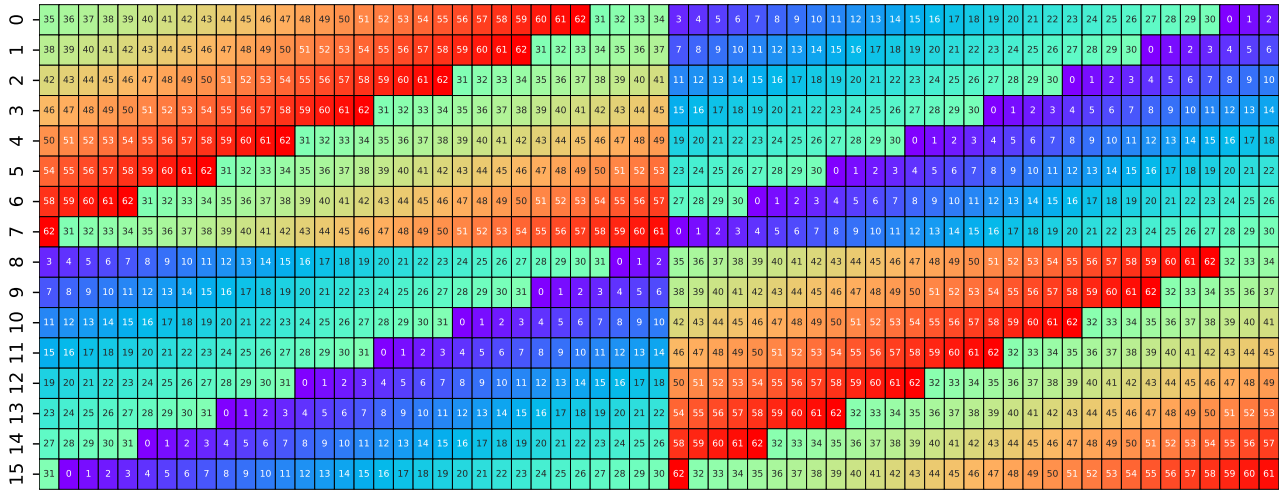


Figure 13. Two nodes global view of swizzling for non-perfect tiling. This example is for GEMM+ReduceScatter.

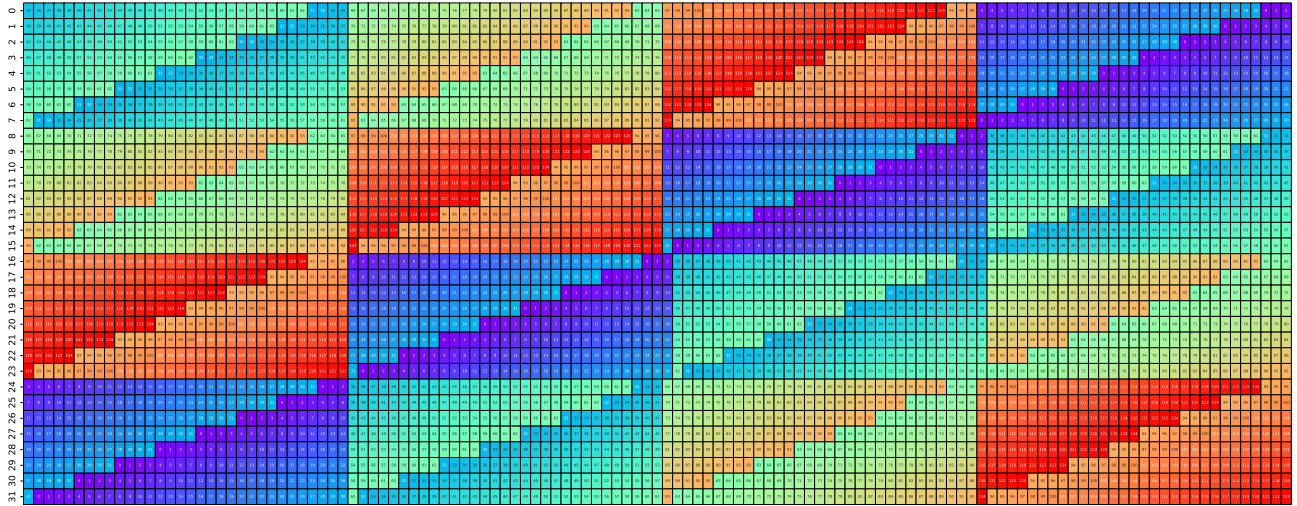


Figure 14. Four nodes global view of swizzling for non-perfect tiling. This example is for GEMM+ReduceScatter.

swizzle logic is implemented in DITRON through efficient kernels. We show the core code for swizzle as follows.

First, swizzle code for AllGather+GEMM. DITRON is implemented based on Triton, so we still use *triton.jit* as compile entry. We extend programming interface with multiple primitives for tile-level or even thread-level control, such as *laneid*, *warp_prefix_sum_kernel*, *__ballot_sync*, *__shfl_sync_i32*.

```

1  @triton.jit(do_not_specialize=["rank"])
2  def threadblock_swizzle_allgather_gemm_kernel(
3      tiled_m,
4      M,
5      rank,
6      WORLD_SIZE: tl.constexpr,
7      NNODES: tl.constexpr,
8      BLOCK_SIZE_M: tl.constexpr,
9      DEBUG: tl.constexpr = False,
10 ):
11     LOCAL_WORLD_SIZE = WORLD_SIZE // NNODES
12     node_id = rank // LOCAL_WORLD_SIZE
13     M_per_rank = M // WORLD_SIZE
14     M_per_node = M // NNODES
15     node_start = node_id
16
17     lane_id = laneid()
18
19     if lane_id < NNODES:
20         n = (lane_id + node_start) % NNODES
21         M_node_start = M_per_node * n
22         M_node_end = M_per_node * (n + 1)
23         tiled_m_node_start = M_node_start // BLOCK_SIZE_M
24         prev_tiled_m_node_end = (M_node_start - 1) // BLOCK_SIZE_M
25         tiled_m_node_end = (M_node_end - 1) // BLOCK_SIZE_M
26         next_tiled_m_node_start = M_node_end // BLOCK_SIZE_M
27
28         if lane_id == 0 and M_node_start != 0:
29             if prev_tiled_m_node_end == tiled_m_node_start:
30                 tiled_m_node_start += 1
31
32         if lane_id == 0 and M_node_end != M:
33             if next_tiled_m_node_start == tiled_m_node_end:
34                 tiled_m_node_end -= 1
35
36         if lane_id != NNODES - 1 and M_node_end != M:
37             if next_tiled_m_node_start == tiled_m_node_end:
38                 tiled_m_node_end -= 1
39         if DEBUG and lane_id == NNODES - 1:
40             print("tiled_m_node_end", tiled_m_node_end)
41
42         swizzled_tiled_m_size = tiled_m_node_end - tiled_m_node_start + 1
43     else:
44         swizzled_tiled_m_size = 0
45
46     if DEBUG and lane_id < NNODES:
47         print("swizzled_tiled_m_size", swizzled_tiled_m_size, lane_id)
48     swizzled_tiled_m_size_accum = (warp_prefix_sum_kernel(swizzled_tiled_m_size, lane_id, NNODES) -
49                                   swizzled_tiled_m_size)
50
51     if DEBUG and lane_id < NNODES:
52         print("swizzled_tiled_m_size_accum", swizzled_tiled_m_size_accum)
53
54     tiled_m_size_l = __shfl_down_sync_i32(0xFFFFFFFF, swizzled_tiled_m_size, NNODES - node_start)
55     tiled_m_size_r = __shfl_up_sync_i32(0xFFFFFFFF, swizzled_tiled_m_size, node_start)
56     tiled_m_size = 0
57     if lane_id < node_start:
58         tiled_m_size = tiled_m_size_l
59     elif lane_id < NNODES:
60         tiled_m_size = tiled_m_size_r
61
62     if DEBUG and lane_id < NNODES:
63         print("tiled_m_size", tiled_m_size)
64
65     tiled_m_size_accum = warp_prefix_sum_kernel(tiled_m_size, lane_id, NNODES) - tiled_m_size
66     mask = __ballot_sync(0xFFFFFFFF, tiled_m < swizzled_tiled_m_size_accum)
67     n = ffs(mask) - 1 - 1
68     if DEBUG and lane_id < NNODES:
69         print("tiled_m_size_accum", tiled_m_size_accum)
70         print("n", n, tiled_m, swizzled_tiled_m_size_accum, mask)
71
72     # map node
73     nid = (n + node_start) % NNODES
74     node_offset = __shfl_sync_i32(0xFFFFFFFF, swizzled_tiled_m_size_accum, n)
75
76     tile_size = __shfl_sync_i32(0xFFFFFFFF, swizzled_tiled_m_size, n)
77
78     tiled_m_intra_node = tiled_m - node_offset
79     local_rank = rank % LOCAL_WORLD_SIZE
80     m_start = M_per_node * nid + M_per_rank * local_rank
81     tiled_m_start = tl.cdiv(m_start, BLOCK_SIZE_M)
82     swizzled_node_offset = __shfl_sync_i32(0xFFFFFFFF, tiled_m_size_accum, nid)
83     rank_offset = max(0, tiled_m_start - swizzled_node_offset) # this may < 0, bad
84     # map rank

```

```

85 tiled_m_intra_node_new = (tiled_m_intra_node + rank_offset) % tile_size
86 return swizzled_node_offset + tiled_m_intra_node_new

```

Then, swizzle code for GEMM+ReduceScatter.

```

1 @triton.jit(do_not_specialize=["rank"])
2 def threadblock_swizzle_gemm_reduce_scatter_kernel(
3     tiled_m,
4     M,
5     rank,
6     WORLD_SIZE: tl.constexpr,
7     NNODES: tl.constexpr,
8     BLOCK_SIZE_M: tl.constexpr,
9     DEBUG: tl.constexpr = False,
10 ):
11     LOCAL_WORLD_SIZE = WORLD_SIZE // NNODES
12     node_id = rank // LOCAL_WORLD_SIZE
13     M_per_rank = M // WORLD_SIZE
14     M_per_node = M // NNODES
15     node_start = node_id + 1
16
17     lane_id = laneid()
18
19     if lane_id < NNODES:
20         n = (lane_id + node_start) % NNODES
21         M_node_start = M_per_node * n
22         M_node_end = M_per_node * (n + 1)
23         tiled_m_node_start = M_node_start // BLOCK_SIZE_M
24         # if tiled_m_node_start overlaps with previous node, then we need to add 1 to tiled_m_node_start
25         prev_tiled_m_node_end = (M_node_start - 1) // BLOCK_SIZE_M
26         if lane_id != 0 and M_node_start != 0:
27             if prev_tiled_m_node_end == tiled_m_node_start:
28                 tiled_m_node_start += 1
29
30         tiled_m_node_end = (M_node_end - 1) // BLOCK_SIZE_M
31         next_tiled_m_node_start = M_node_end // BLOCK_SIZE_M
32         if lane_id == NNODES - 1 and M_node_end != M:
33             if next_tiled_m_node_start == tiled_m_node_end:
34                 tiled_m_node_end -= 1
35
36         swizzled_tiled_m_size = tiled_m_node_end - tiled_m_node_start + 1
37     else:
38         swizzled_tiled_m_size = 0
39
40     if DEBUG and lane_id < NNODES:
41         print("swizzled_tiled_m_size", swizzled_tiled_m_size, lane_id)
42     swizzled_tiled_m_size_accum = (warp_prefix_sum_kernel(swizzled_tiled_m_size, lane_id, NNODES) -
43                                   swizzled_tiled_m_size)
44
45     if DEBUG and lane_id < NNODES:
46         print("swizzled_tiled_m_size_accum", swizzled_tiled_m_size_accum)
47     # thread 0 hold node 'node_start' size
48     # thread 1 hold node 'node_start + 1' size
49     # ...
50     # thread NNODES - node_start hold node '0' size
51     # thread NNODES - 1 hold NNODES 'node_start - 1' size
52
53     # => thread 0 want to hold node 0 size
54     tiled_m_size_l = __shfl_down_sync_i32(0xFFFFFFFF, swizzled_tiled_m_size, NNODES - node_start)
55     tiled_m_size_r = __shfl_up_sync_i32(0xFFFFFFFF, swizzled_tiled_m_size, node_start)
56     tiled_m_size = 0
57     if lane_id < node_start:
58         tiled_m_size = tiled_m_size_l
59     elif lane_id < NNODES:
60         tiled_m_size = tiled_m_size_r
61
62     if DEBUG and lane_id < NNODES:
63         print("tiled_m_size", tiled_m_size)
64
65     tiled_m_size_accum = warp_prefix_sum_kernel(tiled_m_size, lane_id, NNODES) - tiled_m_size
66     mask = __ballot_sync(0xFFFFFFFF, tiled_m < swizzled_tiled_m_size_accum)
67     n = ffs(mask) - 1 - 1
68     if DEBUG and lane_id < NNODES + 1:
69         print("tiled_m_size_accum", tiled_m_size_accum)
70         print("n", n, tiled_m, swizzled_tiled_m_size_accum, mask)
71
72     # map node
73     nid = (n + node_start) % NNODES
74     node_offset = __shfl_sync_i32(0xFFFFFFFF, swizzled_tiled_m_size_accum, n)
75
76     tile_size = __shfl_sync_i32(0xFFFFFFFF, swizzled_tiled_m_size, n)
77
78     tiled_m_intra_node = tiled_m - node_offset
79     local_rank = rank % LOCAL_WORLD_SIZE
80     m_start = M_per_node * nid + M_per_rank * (local_rank + 1)
81     tiled_m_start = m_start // BLOCK_SIZE_M
82     swizzled_node_offset = __shfl_sync_i32(0xFFFFFFFF, tiled_m_size_accum, nid)
83     rank_offset = max(0, tiled_m_start - swizzled_node_offset) # this may < 0, bad
84
85     # map rank

```



```

86 tiled_m_intra_node_new = (tiled_m_intra_node + rank_offset) % tile_size
87 return swizzled_node_offset + tiled_m_intra_node_new

```

At last, the swizzle code for AllGather+MoE.

```

1 @triton.jit(do_not_specialize=["rank"])
2 def threadblock_swizzle_ag_moe_kernel(
3     # input
4     ntokens_by_rank_by_expert_ptr,
5     # output
6     expert_id_ptr,
7     tiled_m_ptr,
8     segment_start_ptr,
9     segment_end_ptr,
10    ntiles_ptr,
11    # workspace buffer
12    ntokens_by_expert_by_rank_acc_ptr,
13    ntiles_by_expert_acc_ptr,
14    ntiles_by_expert_by_stage_ptr,
15    ntiles_by_expert_by_stage_acc_ptr,
16    rank,
17    N_EXPERTS: tl.constexpr,
18    TP_SIZE: tl.constexpr,
19    LOCAL_TP_SIZE: tl.constexpr,
20    NTILES_NEXT_POW_OF_2: tl.constexpr,
21    BLOCK_SIZE_M: tl.constexpr,
22    DEBUG: tl.constexpr = False,
23 ):
24     """
25     tile_index = g(expert_id, stage, index): if tile_index is grouped by expert then stage
26
27     but how to map (stage, index) => tile_index_in_expert?
28
29     first map tile_index_in_expert => (stage, off_in_expert_by_stage, index_in_expert_in_stage)
30
31     tile_index => (expert_id, tile_index_in_expert)           : ntokens grouped by expert_id by rank
32     => (expert_id, stage)                                   : get_stage_id
33     => (expert_id, off_in_expert_by_stage)                  : cumsum by stage
34     => (expert_id, off_in_expert_by_stage, index_in_expert_in_stage) : atomic_add for index_in_expert_in_stage
35     => tile_index_new : by function g
36
37     tile_index -> (expert_id, rank_offset) grouped by (expert_id, rank)
38     rank_offset -> segment_start, segment_end, stage
39
40     so we can remap tile_index as:
41     tile_index_new = g(f(tile_index))
42     """
43     thread_idx = tid(0)
44     N_EXPERTS_NEXT_POW_OF_2: tl.constexpr = next_power_of_2(N_EXPERTS)
45     TP_SIZE_NEXT_POW_OF_2: tl.constexpr = next_power_of_2(TP_SIZE)
46     offs_by_expert = tl.arange(0, N_EXPERTS_NEXT_POW_OF_2)
47     mask_by_expert = offs_by_expert < N_EXPERTS
48     offs_by_rank = tl.arange(0, TP_SIZE_NEXT_POW_OF_2)
49     mask_by_rank = (offs_by_rank < TP_SIZE)
50     offs_by_expert_by_rank = offs_by_expert[:, None] * TP_SIZE + offs_by_rank[None, :]
51     mask_by_expert_by_rank = mask_by_expert[:, None] & mask_by_rank[None, :]
52     offs_by_rank_by_expert = offs_by_rank[:, None] * N_EXPERTS + offs_by_expert[None, :]
53     mask_by_rank_by_expert = mask_by_rank[:, None] & mask_by_expert[None, :]
54     ntokens_by_rank_by_expert = tl.load(ntokens_by_rank_by_expert_ptr + offs_by_rank_by_expert,
55                                         mask=mask_by_rank_by_expert)
56
57     ntokens_by_expert_by_rank = ntokens_by_rank_by_expert.T
58     ntokens_by_expert_by_rank_acc = tl.cumsum(ntokens_by_expert_by_rank, axis=1)
59     ntokens_by_expert = tl.sum(ntokens_by_rank_by_expert, axis=0)
60     ntiles_by_expert = tl.cdiv(ntokens_by_expert, BLOCK_SIZE_M)
61     ntiles_by_expert_acc = tl.cumsum(ntiles_by_expert, axis=0)
62     ntiles = tl.sum(ntiles_by_expert)
63
64     tl.store(ntokens_by_expert_by_rank_acc_ptr + offs_by_expert_by_rank, ntokens_by_expert_by_rank_acc,
65             mask=mask_by_expert_by_rank)
66     tl.store(ntiles_by_expert_acc_ptr + offs_by_expert, ntiles_by_expert_acc)
67
68     # # for each tiled_m in expert eid => stage id / segment_start / segment_end / tiled_m
69     tile_index = tl.arange(0, NTILES_NEXT_POW_OF_2)
70     mask_tile_idx = tile_index < ntiles
71
72     __syncthreads()
73     # tile_index -> (expert_id, offset_by_expert, tile_index_in_expert) -> stage, segment_start, segment_end
74     expert_id, off_by_expert, off_in_expert = bisect_right_with_offset_kernel(ntiles_by_expert_acc_ptr, tile_index,
75                                     N_EXPERTS)
76
77     stage, segment_start, segment_end = get_tile_stage(
78         off_in_expert,
79         rank,
80         TP_SIZE,
81         LOCAL_TP_SIZE,
82         BLOCK_SIZE_M,
83         ntokens_by_expert_by_rank_acc_ptr + expert_id * TP_SIZE,
84     )
85

```

```

86     # histogram by expert by stage
87     tl.store(ntiles_by_expert_by_stage_ptr + offs_by_expert_by_rank, 0, mask=mask_by_expert_by_rank)
88     __syncthreads()
89     off_in_expert_in_stage = tl.atomic_add(ntiles_by_expert_by_stage_ptr + expert_id * TP_SIZE + stage, 1,
90                                           sem="relaxed", scope="gpu", mask=mask_tile_idx)
91     __syncthreads()
92
93     # do some cumsum
94     ntiles_by_expert_by_stage = tl.load(ntiles_by_expert_by_stage_ptr + offs_by_expert_by_rank,
95                                         mask=mask_by_expert_by_rank, other=0)
96     ntiles_by_expert_by_stage_acc = tl.cumsum(ntiles_by_expert_by_stage, axis=1)
97     __syncthreads()
98     tl.store(
99         ntiles_by_expert_by_stage_acc_ptr + offs_by_expert_by_rank,
100         ntiles_by_expert_by_stage_acc,
101         mask=mask_by_expert_by_rank,
102     )
103     __syncthreads()
104
105     off_in_expert_by_stage = tl.where(
106         stage == 0,
107         0,
108         tl.load(ntiles_by_expert_by_stage_acc_ptr + expert_id * TP_SIZE + stage - 1, mask=mask_tile_idx, other=0),
109     )
110
111     tile_index_by_expert_by_stage = off_by_expert + off_in_expert_by_stage + off_in_expert_in_stage
112     __syncthreads()
113
114     tl.store(expert_id_ptr + tile_index_by_expert_by_stage, expert_id, mask=mask_tile_idx)
115     tl.store(tiled_m_ptr + tile_index_by_expert_by_stage, tile_index, mask=mask_tile_idx)
116     tl.store(segment_start_ptr + tile_index_by_expert_by_stage, segment_start, mask=mask_tile_idx)
117     tl.store(segment_end_ptr + tile_index_by_expert_by_stage, segment_end, mask=mask_tile_idx)
118     thread_idx = tid(0)
119     if thread_idx == 0:
120         tl.store(ntiles_ptr, ntiles)
121     if DEBUG and thread_idx < ntiles:
122         print("expert_id", expert_id)

```

B. Evaluation Setup and Workload Configurations

B.1. Evaluation Setup

The H800 clusters used in our evaluation are equipped with NVLink of uni-directional bandwidth of 200 GB/s. There are 8 NICs in one node and each GPU can communicate with another GPU in other nodes at a uni-directional bandwidth of 50 GB/s. The Hopper 96GB HBM clusters used in our evaluation are equipped with NVLink of uni-directional bandwidth of 450 GB/s. There are 4 NICs in one node and each GPU can communicate with another GPU in other nodes at a uni-directional bandwidth of 25 GB/s.

Table 4. Shapes for AllGather+GEMM on H800.

No.	From Model	M	N	K
1	LLaMA3-7B	8192	11008	4096
2	LLaMA3.1-8B	8192	14336	4096
3	LLaMA3.1-70B	8192	28672	8192
4	LLaMA3.1-405B	8192	53248	16384
5	Mistral-7B	8192	14336	4096
6	Qwen2-72B	8192	29568	8192

Table 5. Shapes for GEMM+ReduceScatter and GEMM+AllReduce on H800.

No.	From Model	M	N	K
1	LLaMA3-7B	8192	4096	11008
2	LLaMA3.1-8B	8192	4096	14336
3	LLaMA3.1-70B	8192	8192	28672
4	LLaMA3.1-405B	8192	16384	53248
5	Mistral-7B	8192	4096	14336
6	Qwen2-72B	8192	8192	29568

Table 6. Shapes for AllGather+MoE and MoE+AllReduce on H800.

No.	From Model	num_tokens	hidden_dim	intermediate_size	num_experts	topk
1	Qwen1.5-MoE-A2.7B	8192	2048	1408	60	4
2	Mixtral-8x7B	8192	14336	4096	8	2
3	Mixtral-8x22B	8192	16384	6144	8	2
4	DeepSeek-MoE	8192	1408	2048	64	6

Table 7. Shapes for strong scaling AllGather+GEMM.

Label	From Model	M	N	K
7B	LLaMA3-7B	32768	11008	4096
8B	LLaMA3.1-8B	32768	14336	4096
32B	Qwen3-32B	32768	25600	5120
36B	Seed-OSS-36B	32768	27648	5120
70B	LLaMA3.1-70B	32768	28672	8192
72B	Qwen2-72B	32768	29568	8192
175B	GPT-3-175B	32768	49152	12288
405B	LLaMA3.1-405B	32768	53248	16384

Table 8. Shapes for strong scaling GEMM+ReduceScatter.

1	From Model	M	N	K
7B	LLaMA3-7B	32768	4096	11008
8B	LLaMA3.1-8B	32768	4096	14336
32B	Qwen3-32B	32768	5120	25600
36B	Seed-OSS-36B	32768	5120	27648
70B	LLaMA3.1-70B	32768	8192	28672
72B	Qwen2-72B	32768	8192	29568
175B	GPT-3-175B	32768	12288	49152
405B	LLaMA3.1-405B	32768	16384	53248

Table 9. Shapes for strong scaling GEMM+AllToAll.

Label	seqlen	heads	head_size
case 1	262144	64	128
case 2	491520	64	128
case 3	262144	32	128
case 4	491520	32	128
case 5	262144	16	128
case 6	491520	16	128

B.2. Workload Configuration

The input shapes for AllGather+GEMM used in Figure 4 are listed in Table 4. The input shapes for GEMM+ReduceScatter and GEMM+AllReduce used in Figure 4 are listed in Table 5. The input shapes for AllGather+MoE and MoE+AllReduce used in Figure 4 are listed in Table 6.

For scaling experiments, the shapes for strong scaling AllGather+GEMM are shown in Table 7. The shapes remain the same from 8 GPUs to 32 GPUs. The shapes for weak scaling are the same as those in strong scaling except that the value of M varies with the number of GPUs. We use $M = 8192$ for 8 GPUs, $M = 16384$ for 16 GPUs, and $M = 32768$ for 32 GPUs. The shapes for strong scaling GEMM+ReduceScatter are shown in Table 8. The shapes remain the same for 8 GPUs and 32 GPUs. For weak scaling, the configurations for M are the same as weak scaling AllGather+GEMM.

The shapes for strong scaling GEMM+AllToAll are shown in Table 9. The shapes remain the same from 8 GPUs to 128 GPUs. The shapes for weak scaling are the same as those in strong scaling except that the seqlen varies with the number of GPUs. For 8 GPUs, seqLens are 65536 and 122880; for 16 GPUs, they are 131072 and 245760; for 32 GPUs, they are 262144 and 491520; for 64 GPUs, they are 524288 and 983040; for 128 GPUs, they are 1048576 and 1966080.

C. Detailed Experiment Results

C.1. Detailed Evaluation Results for MoE

The detailed results for AllGather+MoE in Figure 4 are shown in Table 10. The detailed results for MoE+AllReduce in Figure 4 are shown in Table 11. The shapes for each case are shown in Table 6.

Table 10. Latency of AllGather+MoE (ms).

	CuBLAS+NCCL	TileLink	COMET	DITRON
1	18.12	0.56	0.33	0.32
2	7.12	2.14	1.48	1.51
3	8.49	3.30	1.85	1.72
4	24.27	0.65	0.28	0.24

Table 11. Latency of MoE+AllReduce (ms).

	CuBLAS+NCCL	TileLink	DITRON
1	22.04	1.44	0.445
2	7.99	3.70	2.603
3	8.97	4.32	3.289
4	28.64	0.84	0.441

Table 12. Strong scaling results (speedup to CuBLAS+NCCL) for AllGather+GEMM on H800.

From Model	1 Node	2 Node	4 Node
LLaMA3-7B	1.31	1.11	0.63
LLaMA3.1-8B	1.35	1.12	0.61
Qwen3-32B	1.59	1.32	0.65
Seed-OSS-36B	1.64	1.32	0.64
LLaMA3.1-70B	1.67	1.36	0.77
Qwen2-72B	1.67	1.39	1.03
GPT-3-175B	1.46	1.68	1.00
LLaMA3.1-405B	1.44	1.76	1.02

C.2. Detailed Evaluation Results for Scaling

In Figure 7 we have shown strong scaling results for AllGather+GEMM on Hopper 96GB GPUs. Here we show strong scaling results on H800 and weak scaling results on both Hopper 96GB GPUs and H800 in Table 12, Table 13, and Table 14.

Table 13. Weak scaling results (speedup to CuBLAS+NCCL) for AllGather+GEMM on Hopper 96GB HBM GPUs

From Model	1 Node	2 Node	4 Node
LLaMA3-7B	1.23	1.50	0.80
LLaMA3.1-8B	1.19	1.50	0.86
Qwen3-32B	1.06	1.22	0.89
Seed-OSS-36B	1.05	1.26	0.89
LLaMA3.1-70B	1.10	1.29	0.93
Qwen2-72B	1.01	1.18	1.36
GPT-3-175B	1.05	1.23	1.09
LLaMA3.1-405B	1.05	1.19	1.08

Table 14. Weak scaling results (speedup to CuBLAS+NCCL) for AllGather+GEMM on H800

From Model	1 Node	2 Node	4 Node
LLaMA3-7B	1.30	0.98	0.63
LLaMA3.1-8B	1.30	1.02	0.61
Qwen3-32B	1.53	1.14	0.65
Seed-OSS-36B	1.57	1.15	0.64
LLaMA3.1-70B	1.61	1.25	0.77
Qwen2-72B	1.66	1.29	1.03
GPT-3-175B	1.43	1.58	1.00
LLaMA3.1-405B	1.34	1.63	1.02

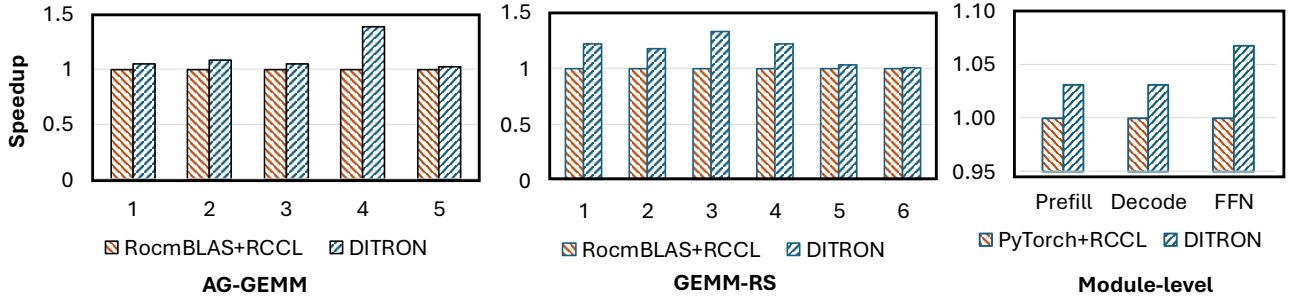


Figure 15. Evaluation results on 8x AMD GPUs. AllGather+GEMM and GEMM+ReduceScatter shapes are in Table 4 and Table 5. Module-level evaluation uses configurations from Qwen3-32B.

C.3. Evaluation Results on AMD GPU

DITRON can be deployed to AMD GPUs through RocSHMEM (AMD) and Triton for AMD. For evaluation, we use shapes from Table 4 and Table 5. Our baseline is RocmBLAS+RCCL. Overall, the speedup ranges from $1.02\times$ to $1.38\times$. The geometric speedup to baseline is $1.11\times$ for AllGather+GEMM and $1.16\times$ to GEMM+ReduceScatter. For module-level evaluation, we use Qwen3-32B. The speedup for prefill attention is $1.03\times$, the speedup for decode attention is $1.03\times$, and the speedup for FFN is $1.07\times$.

C.4. Evaluation Results on PCIe GPU

We evaluate 6 different workloads on PCIe GPUs, including GEMM+ReduceScatter (Table 15), AllToAll (Table 16), AllGather+MoE (Table 17), MoE+ReduceScatter (Table 18), MoE+AllReduce (Table 19), and AllToAll+GEMM (Table 20). The geometric mean speedup of all the workloads is $8.33\times$.

Table 15. GEMM+RS performance on 8×PCIe GPUs.

M	N	K	CuBLAS+NCCL	DITRON	Speedup
8192	8192	29568	18.63 ms	7.34 ms	2.54×

Table 16. All2All performance on 8×PCIe GPU.

num_tokens	hidden_size	num_experts	topk	NCCL	DITRON	speedup
8	3584	128	8	0.42 ms	0.09 ms	4.51×

Table 17. AG+MoE performance on 8×PCIe GPU (shape configurations see Table 6).

No.	num_tokens	CuBLAS+NCCL	DITRON	speedup
1	2048	21.10 ms	0.58 ms	36.26×
2	2048	43.91 ms	2.67 ms	16.47×
3	2048	51.11 ms	3.44 ms	14.86×
4	2048	21.43 ms	0.43 ms	49.84×

Table 18. MoE+RS performance on 8×PCIe GPU.

num_tokens	hidden_size	intermediate_size	num_experts	topk	CuBLAS+NCCL	DITRON	speedup
8192	2048	1536	32	2	15.80 ms	1.71 ms	9.26×

Table 19. MoE+AR performance on 8×PCIe GPU.

num_tokens	hidden_size	intermediate_size	num_experts	topk	CuBLAS+NCCL	DITRON	speedup
8192	2048	1536	32	2	18.19 ms	6.54 ms	2.78×

Table 20. A2A+GEMM performance on 8×PCIe GPU.

DType	M	N	K	CuBLAS+NCCL	DITRON	speedup
Int8	7168	9216	3072	7.11 ms	2.15 ms	3.31×
FP8	7168	9216	3072	8.06 ms	2.17 ms	3.71×

D. Code Examples

In this section, we provide some simplified code examples, including All-Gather GEMM, GEMM Reduce-Scatter and GEMM All-Reduce. For each example, the code is generally divided into three main parts: the host-side launcher, the producer kernel, and the consumer kernel.

D.1. All-Gather GEMM

Here we provide the code example for intra-node All-Gather GEMM. The following is the host-side Python function for launching the All-Gather GEMM operation. It orchestrates the producer (communication) and consumer (computation) kernels.

```

1 def ag_gemm(a, b, ctx: AllGatherGEMMTensorParallelContext, use_cooperative=True):
2     """allgather gemm
3     Allgather global matrix A and do matmul with local matrix B, produces local matrix C
4
5     Args:
6         a (torch.Tensor<float>): local matmul A matrix. shape: [M_per_rank, K]
7         b (torch.Tensor<float>): local matmul B matrix. shape: [N_per_rank, K]
8         ctx: (AllGatherGEMMTensorParallelContext, Optional): if not provided, created immediately
9
10    Returns:
11        C (torch.Tensor<float>): local matmul C matrix. shape: [M, N_per_rank]
12    """
13
14    assert a.shape[1] == b.shape[
15        1], f"tensor_B should has shape (col_major) [{b.shape[0]}, {a.shape[1]}], but get [{b.shape}]"
16    assert a.dtype == b.dtype, f"Dtype of input and weight must be same: tensor_A dtype {a.dtype}, tensor_B dtype {b.dtype}"
17
18    M_per_rank, K = a.shape
19    N_per_rank, _ = b.shape
20
21    assert a.shape[0] * ctx.num_ranks <= ctx.max_M and a.shape[1] == ctx.K, f"Shape of tensor_A must not exceed the maxmize M
22    ↳ of ctx: tensor_A shape [{a.shape}], ctx shape [{ctx.max_M},{ctx.K}]"
23    assert b.shape[0] == ctx.N_per_rank, f"N_per_rank of tensor_B must match that of ctx: tensor_B shape [{b.shape[0]}], ctx
24    ↳ shape [{ctx.N_per_rank}]"
25    assert ctx.tensor_dtype == a.dtype, f"dtype of ctx must match that of ctx: tensor_A dtype {a.dtype}, ctx dtype
26    ↳ {ctx.tensor_dtype}"
27
28    C = torch.empty([ctx.num_ranks * M_per_rank, N_per_rank], dtype=a.dtype, device=a.device)
29
30    local_copy_and_barrier_all(ctx.local_rank, ctx.rank, ctx.num_ranks, a, ctx.symm_workspace, ctx.symm_comm_buf,
31    ↳ ctx.symm_barrier, M_per_rank, K, ctx.phase, is_intranode=False, use_cooperative=use_cooperative)
32    ctx.phase += 2
33
34    current_stream = torch.cuda.current_stream()
35    ctx.ag_intranode_stream.wait_stream(current_stream)
36
37    cp_engine_producer_all_gather_full_mesh_pull(ctx.rank, ctx.num_ranks, a, ctx.symm_workspaces, ctx.symm_barriers,
38    ↳ ctx.ag_intranode_stream, for_correctness=ctx.for_correctness, all_gather_method=ctx.all_gather_method)
39
40    M_per_rank, K = a.shape
41    M = M_per_rank * ctx.num_ranks
42    grid = lambda META: (triton.cdiv(M, META["BLOCK_SIZE_M"]) * triton.cdiv(ctx.N_per_rank, META["BLOCK_SIZE_N"]), )
43    kernel_consumer_gemm[grid](ctx.symm_workspace[:M], b, C, M, ctx.N_per_rank, ctx.K, ctx.symm_workspace.stride(0),
44    ↳ ctx.symm_workspace.stride(1), b.stride(1), b.stride(0), c.stride(0), C.stride(1), ctx.rank, ctx.num_ranks,
45    ↳ ctx.symm_barrier, ctx.BLOCK_M, ctx.BLOCK_N, ctx.BLOCK_K, ctx.GROUP_SIZE_M, num_stages=ctx.stages, num_warps=ctx.warps)
46    current_stream.wait_stream(ctx.ag_intranode_stream)
47
48    return C

```

The producer kernel below is responsible for the All-Gather communication. It copies the local data tensor to the symmetric memory (remote_tensor_buffers) of other GPUs and then sets a signal (_set_signal_cuda) to notify the consumer kernels that the data is ready.

```

1 def cp_engine_producer_all_gather_full_mesh_pull(
2     rank,
3     num_ranks,
4     local_tensor: torch.Tensor,
5     remote_tensor_buffers: List[torch.Tensor],
6     barrier_buffers: List[torch.Tensor],
7     stream: torch.cuda.Stream,
8 ):
9     M_per_rank, N = local_tensor.shape
10
11     rank_orders = [(rank + i) % num_ranks for i in range(num_ranks)]
12
13     with torch.cuda.stream(stream):
14         for src_rank in rank_orders:
15             if src_rank == rank:
16                 continue

```

```

17     dst = remote_tensor_buffers[rnk][src_rnk * M_per_rank:(src_rnk + 1) * M_per_rank, :]
18     src = remote_tensor_buffers[src_rnk][src_rnk * M_per_rank:(src_rnk + 1) * M_per_rank, :]
19     dst.copy_(src)
20     .set_signal_cuda(barrier_buffers[rnk][src_rnk], 1, stream)

```

The consumer kernel first waits on a signal (`dl.wait`) indicating that the required data are available. Once the data is ready, it proceeds with the GEMM computation on the data.

```

1  @triton.jit(do_not_specialize=["rank"])
2  def kernel_consumer_gemm(
3      a_ptr, b_ptr, c_ptr,
4      M, N, K,
5      stride_am, stride_ak,
6      stride_bk, stride_bn,
7      stride_cm, stride_cn, rank, WORLD_SIZE: tl.constexpr, barrier_ptr,
8      BLOCK_SIZE_M: tl.constexpr, BLOCK_SIZE_N: tl.constexpr, BLOCK_SIZE_K: tl.constexpr, #
9      GROUP_SIZE_M: tl.constexpr, #
10 ):
11     a_dtype = a_ptr.dtype.element_ty
12     b_dtype = b_ptr.dtype.element_ty
13     c_dtype = c_ptr.dtype.element_ty
14     tl.static_assert(a_dtype == b_dtype, "A and B must have the same dtype")
15
16     pid = tl.program_id(axis=0)
17     num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
18     num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
19     num_pid_in_group = GROUP_SIZE_M * num_pid_n
20     group_id = pid // num_pid_in_group
21     first_pid_m = group_id * GROUP_SIZE_M
22     group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
23     pid_m = first_pid_m + (pid % num_pid_in_group) % group_size_m
24     pid_n = (pid % num_pid_in_group) // group_size_m
25
26     m_per_rank = M // WORLD_SIZE
27     m_offset = m_per_rank * rank
28     pid_m_offset = tl.cdiv(m_offset, BLOCK_SIZE_M)
29     pid_m = (pid_m + pid_m_offset) % num_pid_m
30
31     offs_am = pid_m * BLOCK_SIZE_M
32     rank_beg = offs_am // m_per_rank
33     rank_end = (min(offs_am + BLOCK_SIZE_M, M) - 1) // m_per_rank
34     token = dl.wait(barrier_ptr + rank_beg, rank_end - rank_beg + 1, 'gpu ', 'acquire ', waitValue=1)
35
36     offs_am = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
37     offs_bn = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
38     offs_k = tl.arange(0, BLOCK_SIZE_K)
39     a_ptrs = a_ptr + (offs_am[:, None] * stride_am + offs_k[None, :] * stride_ak)
40     b_ptrs = b_ptr + (offs_k[:, None] * stride_bk + offs_bn[None, :] * stride_bn)
41
42     a_ptrs = dl.consume_token(a_ptrs, token)
43
44     if a_dtype == tl.int8:
45         accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.int32)
46     else:
47         accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
48     for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):
49         a = tl.load(a_ptrs, mask=offs_k[None, :] < K - k * BLOCK_SIZE_K, other=0.0)
50         b = tl.load(b_ptrs, mask=offs_k[:, None] < K - k * BLOCK_SIZE_K, other=0.0)
51         accumulator += tl.dot(a, b)
52         a_ptrs += BLOCK_SIZE_K * stride_ak
53         b_ptrs += BLOCK_SIZE_K * stride_bk
54
55     offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
56     offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
57     c_ptrs = c_ptr + stride_cm * offs_cm[:, None] + stride_cn * offs_cn[None, :]
58     c_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
59
60     tl.store(c_ptrs, accumulator.to(c_dtype), mask=c_mask)

```

D.2. GEMM Reduce-Scatter

Here we provide the code example for GEMM Reduce-Scatter. The following is the host-side Python function for launching the GEMM Reduce-Scatter operation.

```

1  def gemm_rs_op(input, weight, ctx: GEMMReduceScatterTensorParallelContext, persistent: bool = True,
2                 fuse_scatter: bool = False):
3      world_size = ctx.rs_ctx.world_size
4      local_world_size = ctx.rs_ctx.local_world_size
5      rs_stream = ctx.rs_stream
6      output_dtype = ctx.output_dtype
7      num_gemm_sms = ctx.num_gemm_sms
8

```

```

9     M, local_K = input.shape
10    N = weight.shape[0]
11    assert N == ctx.rs_ctx.N
12
13    assert M % world_size == 0
14    assert weight.shape[1] == local_K
15    M_per_rank = M // world_size
16    current_stream = torch.cuda.current_stream()
17    rs_stream.wait_stream(current_stream)
18
19    output = torch.empty((M_per_rank, N), dtype=output_dtype, device=input.device)
20    workspace = torch.zeros((world_size, ), dtype=torch.int32, device=input.device)
21    gemm_out = ctx.get_gemm_out_buf(input)
22    scatter_signal = ctx.rs_ctx.scatter_signal_buf
23
24    triton_config = triton.Config(
25        {
26            "BLOCK_SIZE_M": ctx.BLOCK_M, "BLOCK_SIZE_N": ctx.BLOCK_N, "BLOCK_SIZE_K": ctx.BLOCK_K, "GROUP_SIZE_M": ctx.GROUP_M
27        }, num_stages=ctx.stages, num_warps=8)
28    triton_config = update_triton_config(M, N, local_K, input.dtype, world_size, local_world_size, triton_config)
29    gemm_rs_producer(input, weight, gemm_out, scatter_signal, workspace, world_size, local_world_size, fuse_scatter,
30    ↪ triton_config)
31
32    if not fuse_scatter:
33        with torch.cuda.stream(rs_stream):
34            reduce_scatter_2d_op(gemm_out, ctx.rs_ctx, output)
35        current_stream.wait_stream(rs_stream)
36    else:
37        nvshmem_barrier_all_on_stream(current_stream)
38        ring_reduce(gemm_out, output, ctx.rs_ctx.local_rank, local_world_size)
39        nvshmem_barrier_all_on_stream(current_stream)
40    return output

```

The producer kernel below is responsible for the GEMM computation. When FUSE_SCATTER is enabled, it directly writes the output to the symmetric memory of the destination rank.

```

1  @triton.jit(launch_metadata=_matmul_launch_metadata, repr=_gemm_rs_non_persistent_repr)
2  def kernel_gemm_rs_producer(
3      # Pointers to matrices
4      a_ptr, # [M, K]_Ti
5      b_ptr, # [K, N]_Ti
6      c_ptr, # [M, N]_To
7      # Matrix dimensions
8      M,
9      N,
10     K,
11     # The stride variables represent how much to increase the ptr by when moving by 1
12     # element in a particular dimension. E.g. 'stride_am' is how much to increase 'a_ptr'
13     # by to get the element one row down (A has M rows).
14     stride_am,
15     stride_ak, #
16     stride_bk,
17     stride_bn, #
18     stride_cm,
19     stride_cn,
20     barrier_ptr,
21     counter_ptr,
22     FUSE_SCATTER: tl.constexpr,
23     LOCAL_WORLD_SIZE: tl.constexpr,
24     WORLD_SIZE: tl.constexpr,
25     # Meta-parameters
26     BLOCK_SIZE_M: tl.constexpr,
27     BLOCK_SIZE_N: tl.constexpr,
28     BLOCK_SIZE_K: tl.constexpr, #
29     GROUP_SIZE_M: tl.constexpr,
30 ):
31     """Kernel for computing the matmul C = A x B.
32     A has shape (M, K), B has shape (K, N) and C has shape (M, N)
33     """
34     tl.static_assert(a_ptr.dtype.is_ptr(), "A should be a pointer")
35     tl.static_assert(b_ptr.dtype.is_ptr(), "B should be a pointer")
36     tl.static_assert(c_ptr.dtype.is_ptr(), "C should be a pointer")
37     a_dtype = a_ptr.dtype.element_ty
38     b_dtype = b_ptr.dtype.element_ty
39     tl.static_assert(a_dtype == b_dtype, "A and B should have the same dtype")
40
41     rank = tl.rank()
42     NNODES = WORLD_SIZE // LOCAL_WORLD_SIZE
43
44     pid = tl.program_id(axis=0)
45     num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
46     num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
47
48     M_per_rank = M // WORLD_SIZE
49     pid_m, pid_n = swizzle_2d(pid, num_pid_m, num_pid_n, GROUP_SIZE_M)
50
51     if NNODES != 1: # with complex threadblock swizzle logic
52         pid_m = threadblock_swizzle_gemm_reduce_scatter_kernel(pid_m, M, rank, WORLD_SIZE, NNODES, BLOCK_SIZE_M)
53     else:

```



```

54     pid_m_offset = (rank + 1) * M_per_rank // BLOCK_SIZE_M
55     pid_m = (pid_m + pid_m_offset) % num_pid_m
56
57     # Create pointers for the first blocks of A and B.
58     offs_am = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
59     offs_bn = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
60     offs_k = tl.arange(0, BLOCK_SIZE_K)
61     a_ptrs = a_ptr + (offs_am[:, None] * stride_am + offs_k[None, :] * stride_ak)
62     b_ptrs = b_ptr + (offs_k[:, None] * stride_bk + offs_bn[None, :] * stride_bn)
63
64     # Iterate to compute a block of the C matrix.
65     if a_ptr.dtype.element_ty == tl.int8:
66         accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.int32)
67     else:
68         accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
69
70     for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):
71         # Load the next block of A and B, generate a mask by checking the K dimension.
72         a = tl.load(a_ptrs, mask=offs_k[None, :] < K - k * BLOCK_SIZE_K, other=0.0)
73         b = tl.load(b_ptrs, mask=offs_k[:, None] < K - k * BLOCK_SIZE_K, other=0.0)
74         # We accumulate along the K dimension.
75         accumulator += tl.dot(a, b)
76         # Advance the ptrs to the next K block.
77         a_ptrs += BLOCK_SIZE_K * stride_ak
78         b_ptrs += BLOCK_SIZE_K * stride_bk
79
80     # Write back the block of the output matrix C with masks.
81     offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
82     offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
83     c_ptrs = c_ptr + stride_cm * offs_cm[:, None] + stride_cn * offs_cn[None, :]
84     out_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
85
86     if not FUSE_SCATTER:
87         tl.store(c_ptrs, accumulator, mask=out_mask)
88
89     # inc barrier
90     segment_start = pid_m * BLOCK_SIZE_M // M_per_rank
91     segment_end = (min((pid_m + 1) * BLOCK_SIZE_M, M) - 1) // M_per_rank
92     __syncthreads()
93     segment = segment_start + tid(axis=0)
94     if segment <= segment_end:
95         m_start = M_per_rank * segment
96         m_end = M_per_rank * (segment + 1) - 1
97         tiled_m_start = m_start // BLOCK_SIZE_M
98         tiled_m_end = m_end // BLOCK_SIZE_M
99         tiled_m_size = tiled_m_end - tiled_m_start + 1
100         val = atomic_add(counter_ptr + segment, 1, semantic="release", scope="gpu")
101         if val == num_pid_n * tiled_m_size - 1:
102             # or use other signal op semantic
103             st(barrier_ptr + segment, 1, scope='gpu ', semantic='release ')
104     else:
105         rank_start = pid_m * BLOCK_SIZE_M // M_per_rank
106         rank_end = (min((pid_m + 1) * BLOCK_SIZE_M, M) - 1) // M_per_rank
107         for cur_rank in range(rank_start, rank_end + 1):
108             m_start = max(M_per_rank * cur_rank, pid_m * BLOCK_SIZE_M)
109             m_end = min(M_per_rank * (cur_rank + 1) - 1, (min((pid_m + 1) * BLOCK_SIZE_M, M) - 1))
110             remote_c_ptr = dl.syemm_at(c_ptr, cur_rank)
111             mask_offset = m_start - pid_m * BLOCK_SIZE_M
112             remote_offs_cm = m_start % M_per_rank + rank * M_per_rank + tl.arange(0, BLOCK_SIZE_M) - mask_offset
113             remote_c_ptrs = remote_c_ptr + stride_cm * remote_offs_cm[:, None] + stride_cn * offs_cn[None, :]
114             remote_mask = (offs_cm[:, None] <= m_end) & (offs_cm[:, None] >= m_start) & (offs_cn[None, :] < N)
115             tl.store(remote_c_ptrs, accumulator, mask=remote_mask)

```

The consumer kernel performs the reduce-scatter communication operation. After the GEMM computation is complete, this kernel reduces the partial results across ranks and scatters them to produce the final output.

```

1 def reduce_scatter_multi_node(input: torch.Tensor, ctx: ReduceScatter2DContext, output: Optional[torch.Tensor] = None):
2     """
3     A hierarchical reduce-scatter implementation that overlaps the intra-node scatter
4     with the local reduce and the inter-node p2p(after reduce). It also provides a rank-wise
5     signal and supports overlap with gemm.
6     """
7     M, N = input.shape
8     M_per_rank = M // ctx.world_size
9
10    current_stream = torch.cuda.current_stream()
11    ctx.reduction_stream.wait_stream(current_stream)
12
13    # directly reduce_scatter to output if nnodes == 1
14    out_each_node = output if ctx.nnodes == 1 else None
15    if not has_fullmesh_nvlink():
16        rs_result_per_node = reduce_scatter_for_each_node_ring(input, ctx, out_each_node)
17    else:
18        rs_result_per_node = reduce_scatter_for_each_node(input, ctx, out_each_node)
19
20    if ctx.nnodes == 1:
21        return rs_result_per_node
22
23    nvshmem_barrier_all_on_stream(current_stream)

```

```

24     if output is None:
25         output = torch.empty((M_per_rank, N), dtype=input.dtype, device=input.device)
26         ring_reduce(rs_result_per_node, output, ctx.node_id, ctx.nnodes)
27     return output
28
29
30 def reduce_scatter_2d_op(input: torch.Tensor, ctx: ReduceScatter2DContext, output: Optional[torch.Tensor] = None):
31     M, N = input.shape
32     assert input.dtype == ctx.dtype
33     assert ctx.max_M >= M and ctx.N == N
34     assert M % ctx.world_size == 0
35
36     nvshmem_barrier_all_on_stream(torch.cuda.current_stream())
37     output = reduce_scatter_multi_node(input, ctx, output)
38     ctx.reset_barriers()
39     return output

```

D.3. GEMM All-Reduce

Here we provide the example for GEMM All-Reduce. The following is the host-side Python function for launching the All-Reduce GEMM operation.

```

1 def gemm_allreduce_op(ctx: GemmARContext, a, b, gemm_config: triton.Config, copy_to_local=True, USE_MULTIMEM_ST=True):
2
3     current_stream = torch.cuda.current_stream()
4     ar_stream = ctx.ar_stream
5     ar_stream.wait_stream(current_stream)
6
7     M, N = a.shape[0], b.shape[0]
8     # Check constraints.
9     assert a.shape[1] == b.shape[1], "Incompatible dimensions"
10    assert a.dtype == b.dtype, "Incompatible dtypes"
11    symm_c = ctx.get_gemm_out_buf(a, b)
12    symm_ar_out = ctx.symm_ar_out_buf
13    gemm_barrier = ctx.gemm_barrier_buf
14    multi_st_barrier = ctx.multi_st_barrier_buf
15    NUM_COMM_SMS = ctx.NUM_COMM_SMS
16    ar_out = torch.empty((M, N), dtype=a.dtype, device=a.device)
17    BLOCK_SIZE_M = gemm_config.all_kwargs()["BLOCK_SIZE_M"]
18    BLOCK_SIZE_N = gemm_config.all_kwargs()["BLOCK_SIZE_N"]
19    # add mask in 'consumer_all_reduce' can remove this constraint
20    assert N % BLOCK_SIZE_N == 0
21    persistent_gemm_notify(a, b, symm_c, gemm_barrier, gemm_config)
22    with torch.cuda.stream(ar_stream):
23        consumer_all_reduce(symm_c, symm_ar_out, ar_out, gemm_barrier, multi_st_barrier, BLOCK_SIZE_M=BLOCK_SIZE_M,
24                           BLOCK_SIZE_N=BLOCK_SIZE_N, NUM_COMM_SMS=NUM_COMM_SMS, USE_MULTIMEM_ST=USE_MULTIMEM_ST)
25    current_stream.wait_stream(ar_stream)
26    if USE_MULTIMEM_ST and copy_to_local:
27        ar_out.copy_(symm_ar_out.reshape(-1)[:, M * N].reshape(M, N))
28    nvshmem_barrier_all_on_stream(current_stream)
29    if USE_MULTIMEM_ST and not copy_to_local:
30        return symm_ar_out.reshape(-1)[:, M * N].reshape(M, N)
31    return ar_out

```

The producer kernel below is responsible for the GEMM computation. It performs computation and notifies the consumer kernels when tiles are ready for all-reduce.

```

1 @triton.jit(do_not_specialize=[])
2 def kernel_persistent_gemm_notify(a_ptr, b_ptr, c_ptr, gemm_barrier_ptr, #
3
4     M, N, K, #
5     stride_am, stride_ak, #
6     stride_bn, stride_bk, #
7     stride_cm, stride_cn, #
8     BLOCK_SIZE_M: tl.constexpr, #
9     BLOCK_SIZE_N: tl.constexpr, #
10    BLOCK_SIZE_K: tl.constexpr, #
11    GROUP_SIZE_M: tl.constexpr, #
12    NUM_GEMM_SMS: tl.constexpr, #
13    ):
14    start_pid = tl.program_id(axis=0)
15    num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
16    num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
17    k_tiles = tl.cdiv(K, BLOCK_SIZE_K)
18    num_tiles = num_pid_m * num_pid_n
19
20    tile_id_c = start_pid - NUM_GEMM_SMS
21
22    offs_k_for_mask = tl.arange(0, BLOCK_SIZE_K)
23    num_pid_in_group = GROUP_SIZE_M * num_pid_n
24
25    for tile_id in tl.range(start_pid, num_tiles, NUM_GEMM_SMS):
26        pid_m, pid_n = _compute_pid(tile_id, num_pid_in_group, num_pid_m, GROUP_SIZE_M, NUM_GEMM_SMS)
27        start_m = pid_m * BLOCK_SIZE_M

```

```

27     start_n = pid_n * BLOCK_SIZE_N
28     offs_am = start_m + tl.arange(0, BLOCK_SIZE_M)
29     offs_bn = start_n + tl.arange(0, BLOCK_SIZE_N)
30     offs_am = tl.where(offs_am < M, offs_am, 0)
31     offs_bn = tl.where(offs_bn < N, offs_bn, 0)
32     offs_am = tl.max_contiguous(tl.multiple_of(offs_am, BLOCK_SIZE_M), BLOCK_SIZE_M)
33     offs_bn = tl.max_contiguous(tl.multiple_of(offs_bn, BLOCK_SIZE_N), BLOCK_SIZE_N)
34
35     accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
36     for ki in range(k_tiles):
37         offs_k = ki * BLOCK_SIZE_K + tl.arange(0, BLOCK_SIZE_K)
38         a_ptrs = a_ptr + (offs_am[:, None] * stride_am + offs_k[None, :] * stride_ak)
39         b_ptrs = b_ptr + (offs_bn[:, None] * stride_bn + offs_k[None, :] * stride_bk)
40
41         a = tl.load(a_ptrs, mask=offs_k_for_mask[None, :] < K - ki * BLOCK_SIZE_K, other=0.0)
42         b = tl.load(b_ptrs, mask=offs_k_for_mask[None, :] < K - ki * BLOCK_SIZE_K, other=0.0)
43         accumulator = tl.dot(a, b.T, accumulator)
44
45     tile_id_c += NUM_GEMM_SMS
46     pid_m, pid_n = _compute_pid(tile_id_c, num_pid_in_group, num_pid_m, GROUP_SIZE_M, NUM_GEMM_SMS)
47     offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
48     offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
49     c_ptrs = c_ptr + stride_cm * offs_cm[:, None] + stride_cn * offs_cn[None, :]
50     c_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
51     c = accumulator.to(c_ptr.dtype.element_ty)
52     tl.store(c_ptrs, c, mask=c_mask)
53     __syncthreads()
54
55     thread_idx = tid(0)
56     gemm_barrier_idx = pid_m * num_pid_n + pid_n
57     if thread_idx == 0:
58         st(gemm_barrier_ptr + gemm_barrier_idx, 1, scope='gpu ', semantic='release ')

```

The consumer kernel performs the all-reduce operation. It waits for GEMM tiles to be ready, then performs reduction across all ranks.

```

1  @triton.jit(do_not_specialize=[])
2  def consumer_all_reduce_kernel(
3      symm_input_ptr,
4      symm_ar_out_ptr,
5      ar_out_ptr, #
6      gemm_barrier_ptr,
7      multi_st_barrier_ptr, #
8      M,
9      N, #
10     BLOCK_SIZE_M: tl.constexpr, #
11     BLOCK_SIZE_N: tl.constexpr, #
12     NUM_COMM_SMS: tl.constexpr,
13     USE_MULTIMEM_ST: tl.constexpr,
14 ):
15     rank = dl.rank()
16     world_size = dl.num_ranks()
17     pid = tl.program_id(0)
18     num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
19     num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
20     num_tiles = num_pid_m * num_pid_n
21     thread_idx = tid(0)
22     block_dim = num_warps() * 32
23     VEC_SIZE: tl.constexpr = 128 // tl.constexpr(symm_input_ptr.dtype.element_ty.primitive_bitwidth)
24
25     tl.static_assert(BLOCK_SIZE_N % VEC_SIZE == 0)
26     VEC_PER_ROW = BLOCK_SIZE_N // VEC_SIZE
27     src_data_mc_ptr = libshmem.device.remote_mc_ptr(libshmem.device.NVSHMEMX.TEAM.NODE, symm_input_ptr)
28     if not USE_MULTIMEM_ST:
29         for tile_id in range(pid, num_tiles, NUM_COMM_SMS):
30             pid_m = tile_id // num_pid_n
31             pid_n = tile_id % num_pid_n
32             if thread_idx < world_size:
33                 peer_gemm_barrier_ptr = dl.symm.at(gemm_barrier_ptr, thread_idx)
34                 while ld(peer_gemm_barrier_ptr + tile_id, scope='sys ', semantic='acquire ') != 1:
35                     pass
36                 __syncthreads()
37                 tile_m = min(M - pid_m * BLOCK_SIZE_M, BLOCK_SIZE_M)
38                 cur_tile_nelem = tile_m * BLOCK_SIZE_N
39                 for idx in range(thread_idx, cur_tile_nelem // VEC_SIZE, block_dim):
40                     row_id = idx // VEC_PER_ROW
41                     col_id = idx % VEC_PER_ROW
42                     offset = (row_id + pid_m * BLOCK_SIZE_M) * N + col_id * VEC_SIZE + pid_n * BLOCK_SIZE_N
43                     val0, val1, val2, val3 = multimem_ld_reduce_v4(src_data_mc_ptr + offset)
44                     st_v4_b32(ar_out_ptr + offset, val0, val1, val2, val3)
45     else:
46         symm_out_mc_ptr = libshmem.device.remote_mc_ptr(libshmem.device.NVSHMEMX.TEAM.NODE, symm_ar_out_ptr)
47         for tile_id in range(pid + rank * NUM_COMM_SMS, num_tiles, NUM_COMM_SMS * world_size):
48             pid_m = tile_id // num_pid_n
49             pid_n = tile_id % num_pid_n

```

```

50     if thread_idx < world_size:
51         peer.gemm.barrier_ptr = dl.symm.at(gemm.barrier_ptr, thread_idx)
52         while ld(peer.gemm.barrier_ptr + tile_id, scope="sys", semantic="acquire") != 1:
53             pass
54         __syncthreads()
55
56         tile_m = min(M - pid_m * BLOCK_SIZE_M, BLOCK_SIZE_M)
57         cur_tile_nelem = tile_m * BLOCK_SIZE_N
58         for idx in range(thread_idx, cur_tile_nelem // VEC_SIZE, block_dim):
59             row_id = idx // VEC_PER_ROW
60             col_id = idx % VEC_PER_ROW
61             offset = (row_id + pid_m * BLOCK_SIZE_M) * N + col_id * VEC_SIZE + pid_n * BLOCK_SIZE_N
62             val0, val1, val2, val3 = multimem_ld_reduce_v4(src_data_mc_ptr + offset)
63             multimem_st_v4(symm_out_mc_ptr + offset, val0, val1, val2, val3)
64         __syncthreads()
65
66         # barrier on all blocks with same pid
67         # 0. set barrier to all blocks with same pid on all peer ranks
68         if thread_idx < world_size:
69             peer_ptr = dl.symm.at(multi.st.barrier_ptr, thread_idx)
70             st(peer_ptr + rank * NUM_COMM_SMS + pid, 1, scope='sys ', semantic='release ')
71
72         # 1. wait barrier
73         if thread_idx < world_size:
74             multi.st.barrier_idx = thread_idx * NUM_COMM_SMS + pid
75             while ld(multi.st.barrier_ptr + multi.st.barrier_idx, scope='sys ', semantic='acquire ') != 1:
76                 pass
77             st(multi.st.barrier_ptr + multi.st.barrier_idx, 0)
78
79         # Each block can safely reset the part of the barriers it is waiting for.
80         # In low latency kernel, we ensure that the gemm barriers used for two consecutive iteration are different,
81         # it can be reset without any sync.
82         for tile_id in range(pid + rank * NUM_COMM_SMS, num_tiles, NUM_COMM_SMS * world_size):
83             peer.gemm.barrier_ptr = dl.symm.at(gemm.barrier_ptr, thread_idx)
84             if thread_idx < world_size:
85                 st(peer.gemm.barrier_ptr + tile_id, 0, scope='sys ', semantic='relaxed ')

```

D.4. Register MegaKernel Task

In this section, we use linear (GEMM) as an example to show how to register kernels as tasks. The following code contains a tile-level GEMM kernel, a chunk-level (multiple tiles) GEMM kernel, and an interface kernel to keep unified function interface for all various tasks. These code are written by users and only minor modification is required to change users' original kernels.

```

1  @triton.jit
2  def tile_wise_matmul_compute(tile_id, a_ptr, b_ptr, c_ptr, M, N, K, BLOCK_SIZE_M, BLOCK_SIZE_N, BLOCK_SIZE_K,
3                               NUM_STAGES):
4      # linear: a (M, K) x b (N, K) -> c (M, N)
5      num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
6      k_tiles = tl.cdiv(K, BLOCK_SIZE_K)
7
8      offs_k_for_mask = tl.arange(0, BLOCK_SIZE_K)
9
10     pid_m = tile_id // num_pid_n
11     pid_n = tile_id % num_pid_n
12     start_m = pid_m * BLOCK_SIZE_M
13     start_n = pid_n * BLOCK_SIZE_N
14     offs_am = start_m + tl.arange(0, BLOCK_SIZE_M)
15     offs_bn = start_n + tl.arange(0, BLOCK_SIZE_N)
16     offs_am = tl.where(offs_am < M, offs_am, 0)
17     offs_bn = tl.where(offs_bn < N, offs_bn, 0)
18     offs_am = tl.max_contiguous(tl.multiple_of(offs_am, BLOCK_SIZE_M), BLOCK_SIZE_M)
19     offs_bn = tl.max_contiguous(tl.multiple_of(offs_bn, BLOCK_SIZE_N), BLOCK_SIZE_N)
20     accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
21     for ki in tl.range(0, k_tiles, num_stages=NUM_STAGES):
22         offs_k = ki * BLOCK_SIZE_K + tl.arange(0, BLOCK_SIZE_K)
23         a_ptrs = a_ptr + (offs_am[:, None] * K + offs_k[None, :])
24         b_ptrs = b_ptr + (offs_bn[:, None] * K + offs_k[None, :])
25
26         a = tl.load(a_ptrs, mask=offs_k_for_mask[None, :] < K - ki * BLOCK_SIZE_K, other=0.0)
27         b = tl.load(b_ptrs, mask=offs_k_for_mask[None, :] < K - ki * BLOCK_SIZE_K, other=0.0)
28         accumulator = tl.dot(a, b.T, accumulator)
29
30     offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
31     offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
32     c_ptrs = c_ptr + N * offs_cm[:, None] + offs_cn[None, :]
33     c_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
34     c = accumulator.to(c_ptr.dtype.element_ty)
35     tl.store(c_ptrs, c, mask=c_mask)
36
37
38  @triton.jit

```

```

39 def tile_range_matmul_compute_and_notify(tile_start, sb_base_ptr, a_ptr, b_ptr, c_ptr, M, N, K, BLOCK_SIZE_M,
40                                         BLOCK_SIZE_N, BLOCK_SIZE_K, NUM_STAGES, TILE_READY_SIGNAL, NUM_SMS):
41     num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
42     num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
43     num_tiles = num_pid_m * num_pid_n
44     for tile_id in tl.range(tile_start, num_tiles, NUM_SMS, flatten=True, warp_specialize=True):
45         tile_wise_matmul_compute(tile_id, a_ptr, b_ptr, c_ptr, M, N, K, BLOCK_SIZE_M, BLOCK_SIZE_N, BLOCK_SIZE_K,
46                                 NUM_STAGES)
47         st(sb_base_ptr + tile_id, TILE_READY_SIGNAL, "gpu", "release")
48
49 @triton.jit
50 def linear_task_compute(task_base_info: TaskBaseInfo, scoreboard: Scoreboard, BLOCK_SIZE_M: tl.constexpr,
51                         BLOCK_SIZE_N: tl.constexpr, BLOCK_SIZE_K: tl.constexpr, NUM_STAGES: tl.constexpr,
52                         ALIGNMENT_K: tl.constexpr):
53
54     input: TensorDesc = task_base_info.get_tensor(0)
55     weight: TensorDesc = task_base_info.get_tensor(1)
56     output: TensorDesc = task_base_info.get_tensor(2)
57
58     M = input.size(0)
59     K = input.size(1, ALIGNMENT_K)
60     N = weight.size(0)
61     a_ptr = input.data_ptr(tl.bfloat16)
62     b_ptr = weight.data_ptr(tl.bfloat16)
63     c_ptr = output.data_ptr(tl.bfloat16)
64
65     tile_id = task_base_info.tile_id_or_start
66     tile_wise_matmul_compute(tile_id, a_ptr, b_ptr, c_ptr, M, N, K, BLOCK_SIZE_M, BLOCK_SIZE_N, BLOCK_SIZE_K,
67                             NUM_STAGES)
68     scoreboard.release_tile(task_base_info, tile_id)

```

Then, we can register the kernels as tasks using the following interface. Users are required to describe the problem size, data dependency, and the tiling configurations.

```

1 @registry.register_task(op_type="linear", task_cls=LinearTask, config_factory=linear_config_factory,
2                         codegen_func=codegen_linear)
3 class LinearTaskBuilder(TaskBuilderBase):
4
5     @classmethod
6     def get_problem_size(cls, io_tensors: List[List['torch.Tensor']], extra_params: Dict[str, Any]):
7         a, b = io_tensors[0]
8         M, K = a.shape
9         N, K = b.shape
10        return (M, N, K)
11
12    @classmethod
13    def _build_tasks_impl(cls, device_prop, layer_id: int, dependency: TaskDependency, io_tensors, extra_params,
14                          tile_wise=True, config_args={} ) -> List[TaskBase]:
15        assert tile_wise == True # noqa: E712
16        kernel_config = cls.create_config(**config_args)
17        task_id = cls.get_task_id(layer_id)
18        BLOCK_SIZE_M = kernel_config.BLOCK_SIZE_M
19        BLOCK_SIZE_N = kernel_config.BLOCK_SIZE_N
20        M, N, K = cls.get_problem_size(io_tensors, extra_params)
21        num_tiles_m = cdiv(M, BLOCK_SIZE_M)
22        num_tiles_n = cdiv(N, BLOCK_SIZE_N)
23        num_tiles = num_tiles_m * num_tiles_n
24        x, w = io_tensors[0]
25        y = io_tensors[1][0]
26
27        num_sm = device_prop.NUM_SMS
28        tasks = []
29        cls.log(
30            f"Linear Task: M = {M}, N = {N}, K = {K}, num_tiles = {num_tiles}, num_sm = {num_sm}, tile_wise = {tile_wise},
31            ↳ dependency = {dependency}, BLOCK_SIZE_M={BLOCK_SIZE_M}, BLOCK_SIZE_N = {BLOCK_SIZE_N}"
32        )
33        for tm in range(num_tiles_m):
34            for tn in range(num_tiles_n):
35                tile_id = tm * num_tiles_n + tn
36                bm = min(BLOCK_SIZE_M, M - tm * BLOCK_SIZE_M)
37                bn = min(BLOCK_SIZE_N, N - tn * BLOCK_SIZE_N)
38                x_desc = InputDependencyDesc(x, require_full=False, start_indices=(tm * BLOCK_SIZE_M, 0),
39                                           data_sizes=(bm, K))
40                w_desc = InputDependencyDesc(w, require_full=False, start_indices=(tn * BLOCK_SIZE_N, 0),
41                                           data_sizes=(bn, K))
42                y_desc = OutputTilingDesc(tile_sizes=(BLOCK_SIZE_M, BLOCK_SIZE_N),
43                                         start_indices=(tm * BLOCK_SIZE_M, tn * BLOCK_SIZE_N))
44                inputs_dep = {x: x_desc, w: w_desc}
45                outs_tile_mapping = {y: y_desc}
46                tasks.append(
47                    cls._create_task(layer_id, task_id, tile_id, num_tiles, kernel_config, dependency, io_tensors,
48                                   extra_params, inputs_dep, outs_tile_mapping))
49        return tasks
50
51    @classmethod
52    def build_tasks(cls, device_prop: 'DeviceProp', layer_id: int, dependency: TaskDependency,
53                   io_tensors: List[List['torch.Tensor']], extra_params: Dict[str, Any]) -> List[TaskBase]:
54        return cls._build_tasks_impl(device_prop, layer_id, dependency, io_tensors, extra_params)

```

Using the registration interface, multiple tasks can be added. And DITRON will **automatically** generate MegaKernel. Here we show the generated code. Note that the code is auto-generated, not written by users. The task-level primitives such as `fetch_task` and `scoreboard.wait_deps` are used here.

```

1 @triton.jit
2 def FETCH_TASK(work_queues, idx, INT_PER_TASK, NUM_SMS, MAX_NUM_TENSOR_DIMS, ENABLE_RUNTIME_SCHEDUER=False):
3     sm_id = tl.program_id(axis=0)
4     TASK_TYPE_OFFSET = 0
5     LAYER_ID_OFFSET = 1
6     TASK_ID_OFFSET = 2
7     TILE_ID_OR_START_OFFSET = 3
8     DEPEND_ENTRY_START_OFFSET = 4
9     DEPEND_ENTRY_END_OFFSET = 5
10    IO_TENSORS_OFFSET = 6
11
12    if not ENABLE_RUNTIME_SCHEDUER:
13        offset = INT_PER_TASK * NUM_SMS
14
15        task_type = tl.load(work_queues + idx * offset + sm_id * INT_PER_TASK + TASK_TYPE_OFFSET).to(tl.int32)
16        layer_id = tl.load(work_queues + idx * offset + sm_id * INT_PER_TASK + LAYER_ID_OFFSET).to(tl.int32)
17        task_id = tl.load(work_queues + idx * offset + sm_id * INT_PER_TASK + TASK_ID_OFFSET).to(tl.int32)
18        tile_id_or_start = tl.load(work_queues + idx * offset + sm_id * INT_PER_TASK + TILE_ID_OR_START_OFFSET).to(tl.int32)
19        depend_entry_start = tl.load(work_queues + idx * offset + sm_id * INT_PER_TASK +
20        ↪ DEPEND_ENTRY_START_OFFSET).to(tl.int32)
21        depend_entry_end = tl.load(work_queues + idx * offset + sm_id * INT_PER_TASK + DEPEND_ENTRY_END_OFFSET).to(tl.int32)
22        io_tensors_ptr = work_queues + idx * offset + sm_id * INT_PER_TASK + IO_TENSORS_OFFSET
23    else:
24        task_type = tl.load(work_queues + idx * INT_PER_TASK + TASK_TYPE_OFFSET).to(tl.int32)
25        layer_id = tl.load(work_queues + idx * INT_PER_TASK + LAYER_ID_OFFSET).to(tl.int32)
26        task_id = tl.load(work_queues + idx * INT_PER_TASK + TASK_ID_OFFSET).to(tl.int32)
27        tile_id_or_start = tl.load(work_queues + idx * INT_PER_TASK + TILE_ID_OR_START_OFFSET).to(tl.int32)
28        depend_entry_start = tl.load(work_queues + idx * INT_PER_TASK + DEPEND_ENTRY_START_OFFSET).to(tl.int32)
29        depend_entry_end = tl.load(work_queues + idx * INT_PER_TASK + DEPEND_ENTRY_END_OFFSET).to(tl.int32)
30        io_tensors_ptr = work_queues + idx * INT_PER_TASK + IO_TENSORS_OFFSET
31
32    task_base_info = TaskBaseInfo(io_tensors_ptr, task_type, layer_id, task_id, tile_id_or_start, depend_entry_start,
33    ↪ depend_entry_end, MAX_NUM_TENSOR_DIMS)
34    return task_base_info
35
36 @triton.jit
37 def MEGA_TRITON_KERNEL(
38     work_queue_start, # [1, ] int32, init with zero
39     work_queues, # [MAX_INS, NUM_SMS, INS], int32
40     num_tasks_per_wq, #[num_sms,]
41     scoreboard_ptr,
42     task_deps_ptr, # [num_deps_entry_of_all_tasks, INT_PER_DEPS]
43
44     INT_PER_DEPS: tl.constexpr,
45     INT_PER_TASK: tl.constexpr,
46     MAX_TASK_ID: tl.constexpr,
47     MAX_NUM_TILES_PER_OP: tl.constexpr,
48     MAX_NUM_TENSOR_DIMS: tl.constexpr,
49     NUM_SMS: tl.constexpr,
50     num_warps: tl.constexpr
51 ):
52
53     WARP_SIZE: tl.constexpr = 32
54     NUM_THREADS: tl.constexpr = num_warps * WARP_SIZE
55     scoreboard = Scoreboard(task_deps_ptr, INT_PER_DEPS, scoreboard_ptr, MAX_TASK_ID, MAX_NUM_TILES_PER_OP, tl.constexpr(1),
56     ↪ NUM_THREADS)
57     sm_id = tl.program_id(axis=0)
58
59     num_tasks = tl.load(num_tasks_per_wq + sm_id)
60     cur_task_idx = 0
61
62
63     # early exit
64     if cur_task_idx >= num_tasks:
65         return
66
67     cur_task_base_info = FETCH_TASK(work_queues, cur_task_idx, INT_PER_TASK, NUM_SMS, MAX_NUM_TENSOR_DIMS,
68     ↪ ENABLE_RUNTIME_SCHEDUER=False)
69     nxt_task_base_info = cur_task_base_info
70     nxt_task_idx = cur_task_idx
71
72     while cur_task_idx < num_tasks:
73
74         task_type = cur_task_base_info.task_type
75
76         # task kernel need to set signal for each tile
77
78         scoreboard.wait_deps(cur_task_base_info)
79
80         #### run task ####
81
82
83

```



```

84     task_base_info = cur_task_base_info
85
86     if task_type == 0: # RMSNormTask
87         rmsnorm_task_compute(task_base_info, scoreboard, RMS_EPS=1e-06, BLOCK_SIZE_N = 2048)
88
89     elif task_type == 6: # QKVProjTask
90         fcl_task_compute(task_base_info, scoreboard, BLOCK_SIZE_M=16, BLOCK_SIZE_N=64,
91                         BLOCK_SIZE_K=256, NUM_STAGES=5)
92
93     elif task_type == 9: # QKNormRopeUpdateKVCacheTask
94         rmsnorm_rope_update_kv_cache_task_compute(
95             task_base_info, scoreboard, NUM_Q_HEADS=4, NUM_KV_HEADS=1, Q_HEAD_DIM=128,
96             V_HEAD_DIM=128, PAGE_SIZE=1, MAX_NUM_BLOCKS_PER_SEQ=1024,
97             Q_RMS_EPS=1e-06, K_RMS_EPS=1e-06
98         )
99
100    elif task_type == 2: # AttnSplitTask
101        attn_gqa_fwd_batch_decode_split_kv_task_compute(
102            task_base_info, scoreboard, SM_SCALE=0.08838834764831845, SOFT_CAP=0.0,
103            NUM_Q_HEADS=4, NUM_KV_HEADS=1, Q_HEAD_DIM=128, V_HEAD_DIM=128, PAGE_SIZE=1,
104            MAX_NUM_BLOCKS_PER_SEQ=1024, BLOCK_N=64, BLOCK_HEAD_DIM=128,
105            BLOCK_DPE=0, BLOCK_DV=128, BLOCK_H=16, NUM_KV_SPLITS=32
106        )
107
108    elif task_type == 12: # AttnCombineTask
109        attn_gqa_fwd_batch_decode_combine_task_compute(
110            task_base_info, scoreboard, NUM_Q_HEADS=4, V_HEAD_DIM=128, BLOCK_DV=128, NUM_KV_SPLITS=32
111        )
112
113    elif task_type == 4: # OProjTask
114        fcl_task_compute(task_base_info, scoreboard, BLOCK_SIZE_M=16, BLOCK_SIZE_N=128,
115                        BLOCK_SIZE_K=64, NUM_STAGES=7)
116
117    elif task_type == 11: # BarrierAllIntraNodeTask
118        barrier_all_intra_node_task_compute(task_base_info, scoreboard)
119
120    elif task_type == 7: # AllReduceTask
121        allreduce_task_compute(task_base_info, scoreboard, BLOCK_SIZE=1024)
122
123    elif task_type == 8: # AddTask
124        add_task_compute(task_base_info, scoreboard, BLOCK_SIZE=512)
125
126    elif task_type == 1: # MLPFC1Task
127        fcl_task_compute(task_base_info, scoreboard, BLOCK_SIZE_M=16, BLOCK_SIZE_N=64,
128                        BLOCK_SIZE_K=128, NUM_STAGES=6)
129
130    elif task_type == 3: # SiLUMulUpTask
131        silu_mul_up_task_compute(task_base_info, scoreboard, BLOCK_SIZE_M=8, BLOCK_SIZE_N=128)
132
133    elif task_type == 5: # MLPFC2Task
134        fcl_task_compute(task_base_info, scoreboard, BLOCK_SIZE_M=16, BLOCK_SIZE_N=64,
135                        BLOCK_SIZE_K=256, NUM_STAGES=6)
136
137    elif task_type == 10: # LinearTask
138        linear_task_compute(task_base_info, scoreboard, BLOCK_SIZE_M=16, BLOCK_SIZE_N=128,
139                          BLOCK_SIZE_K=128, NUM_STAGES=4, ALIGNMENT_K=16)
140
141    # next task
142
143    cur_task_idx = cur_task_idx + 1
144    if cur_task_idx < num_tasks:
145        cur_task_base_info = FETCH_TASK(work_queues, cur_task_idx, INT_PER_TASK, NUM_SMS, MAX_NUM_TENSOR_DIMS,
146        ↪  ENABLE_RUNTIME_SCHEDULER=False)
147

```

E. Hardware Primitives

We list the unified primitives in DITRON. These primitives are common among various hardware backends. To port DITRON to different hardware, the developers only need to instantiate these primitives with corresponding code generation rules.

Primitive Name	Parameters	Description
Distributed Primitives		
wait	barrierPtrs, numBarriers, scope, semantic	Waits for a signal on barrier pointer(s).
consume_token	value, token	Enforces ordering dependencies using a token.
rank	axis=-1	Returns the process rank.
num_ranks	axis=-1	Returns the total number of ranks.
symm_at	ptr, rank	Returns a pointer to symmetric memory on a remote rank.
notify	ptr, rank, signal, ...	Sends a signal to a remote rank.
SIMT Primitives		
simt_exec_region	<i>None</i>	Context manager for SIMT execution.
extract	input, indices	Extracts a scalar from a tensor.
insert	input, scalar, indices	Inserts a scalar into a tensor.
vector	args	Creates a vector from values.
zeros_vector	vec_size, dtype	Creates a zero-filled vector.
tid	axis	Thread ID (0=x, 1=y, 2=z).
ntid	axis	Block dimension (number of threads).
laneid	<i>None</i>	Lane ID within a warp.
__syncthreads	<i>None</i>	Synchronizes threads in a block.
atomic_cas	ptr, cmp, val, ...	Atomic Compare-And-Swap.
atomic_add	ptr, val, ...	Atomic Add.
ld/st	ptr, val, scope, semantic	Load/Store with specific memory semantics.
smid	<i>None</i>	Streaming Multiprocessor ID.
SHMEM Device Primitives (NVSHMEM/ROCSHMEM)		
my_pe	<i>None</i>	Returns current PE (Processing Element) ID.
n_pes	<i>None</i>	Returns total number of PEs.
remote_ptr	local_ptr, pe	Resolves a pointer on a remote PE.
barrier_all	<i>None</i>	Synchronizes all PEs.
sync_all	<i>None</i>	Synchronizes all PEs (lighter than barrier).
fence	<i>None</i>	Ensures ordering of memory operations.
putmem	dest, source, bytes, pe	Writes data to remote memory (Blocking).
putmem_nbi	dest, source, bytes, pe	Writes data to remote memory (Non-Blocking).
getmem	dest, source, bytes, pe	Reads data from remote memory (Blocking).
getmem_nbi	dest, source, bytes, pe	Reads data from remote memory (Non-Blocking).
putmem_signal	dest, src, bytes, sig, ...	Writes data and signals remote PE.

Table 21. Unified Primitives in DITRON